ARCHITECT ROADMAP

.NET and Angular

30 days to technical architect

Decisions · Quality attributes · C4 and ADRs · Identity and data

Azure shape · Operability · Frontend boundaries · AI as a critic

For a full-stack developer with about eight years of .NET and Angular delivery who is moving into a technical architect seat.

Prepared 31 October 2026. New work targets .NET 10 LTS, supported through 14 November 2028. .NET 8 and .NET 9 reach end of support in November 2026. For Angular, speak in features you have verified on angular.dev the week you write a standard: standalone components, signals, built-in control flow, and typed forms. Do not memorize a price, a quota, or an Angular major you have not checked.

What 30 days can actually change

Thirty days will not manufacture ten years of architecture tenure. A company still has to give you the seat, and the seat is earned again every quarter by decisions other people can live with. What 30 focused days will do, on top of eight years of shipping .NET and Angular, is close the gap that keeps a strong senior engineer sounding like a senior engineer: you leave with the artifacts, the tradeoff language, and a way of using AI that a technical architect is expected to show.

By the evening of day 30 you should be able to do five things without notes. Draw the context and the containers for a system you know. Write an architecture decision record that names options, a recommendation, consequences, and a revisit trigger. Turn a quality hope such as 'it must be fast and secure' into a scenario with a number and a window. Lead a one-hour design review without taking the pen. Say what you will not split into microservices, and why.

The trap in this title

Technical architect here means the person who sets and reviews the technical shape of a product estate: boundaries, data ownership, identity, integration, operability, and standards, and who can still read the .NET and Angular. It is not enterprise architect, and it is not a slide role that has left the code. Solution architect, in many services firms, is the same engineering with a heavier workshop, estimate, and stakeholder document. This roadmap trains both rooms. It does not train a vendor-only pre-sales role.

How the days are built

Each day is about four hours, 120 hours in total. If you only have two hours, cut the external reading first. Do not cut the artifact and do not cut saying the answer out loud. Reading this document quietly will not change how you sound in a review.

Block	Minutes	What you do
Study	70	Read that day's explanation in this document until you can restate it.
External source	30	Read only the section named for that day. Stop when the timer ends.
Artifact	70	Write the page, diagram, or decision in your own words.
Speak	30	Say the Say this box in under two minutes. Then say the failure mode.
AI critic	30	Run that day's prompt. Reject anything you cannot defend. Edit the artifact.
Redraw	10	Close the laptop picture. Redraw the decision from memory on paper.

Where this sits next to your other guides

The 10-day tech lead roadmap and the deep dive are the depth spike: C#, ASP.NET Core, EF Core, SQL, Azure hosts, and a first microservices pass. The further 20 days cover pipelines, MVC, and AI-assisted delivery. This 30-day document does not repeat those lessons. When a panel drops into a coding or LINQ question, use those guides. When they ask what you decided and what you gave up, use this one.

The 30 days

Day	You practice	Artifact before sleep
1	The seat, and how it differs from senior and tech lead	Positioning line and a stakeholder map
2	Quality attributes and scenarios	Three scenarios for the sample system
3	C4 and a short architecture description	Context and container diagrams
4	Architecture decision records	ADR-001 modular monolith
5	Boundaries, language, and team shape	Context map on one page
6	Angular and .NET reference shape, including the BFF	Request path and auth notes
7	Week review	Principles, one page, edited after an AI critique
8	Integration styles and contracts	Sync versus async table for the sample flows
9	Data ownership, tenancy, outbox	Schema ownership sketch
10	Consistency, duplicates, sagas	Order state machine and one compensation
11	Identity and threat modeling	STRIDE notes on submit order
12	Azure shape and Well-Architected	Host decision table
13	API as a contract	Status-code and idempotency rules
14	Week review	ADR-002 messaging, plus the threat model
15	SLOs, timeouts, retries, bulkheads	SLO sheet
16	Observability from browser to SQL	What you log, measure, and page on
17	Cost and capacity without invented prices	Assumption and cost-driver page
18	Evolutionary architecture and strangler	One migration slice with rollback
19	Angular architecture at this altitude	ADR-003 frontend composition
20	Governance a team will accept	Review triggers and a 15-item radar
21	Week review	Ten-minute architecture briefing, spoken
22	Case: design from a one-page brief	Containers, data, accept path, one failure
23	Case: legacy modernization	One-page plan a director can fund
24	Case: notify, audit, and search	Three stores, three reasons
25	Writing for a stakeholder	Decision memo: why not microservices this half
26	Facilitating a review	Agenda and the questions you will ask
27	Portfolio	README plus five stories from your own career
28	Mock architecture interview	Scored notes from a 45-minute design
29	Repair the weak spot	Second case: a partner API
30	First 90 days in the seat	Listening tour and the first fitness function

Who you are in the review

You have eight years of delivery. You have shipped Angular screens and .NET APIs. You have felt a release go wrong. The architect interview fails when that history comes out as a list of frameworks. It lands when it comes out as a decision: the constraint, the option you rejected, the consequence, and the signal that would make you reopen it.

Positioning line

I have about eight years as a full-stack .NET and Angular developer. I am moving into a technical architect role. I still go deep on the API, the data access, and the SPA, and I now own the shape around them: boundaries, quality attributes, identity, and how we operate the system. I decide with a tradeoff, a consequence, and a revisit trigger. I do not split a system into services because a diagram looks modern.

What changes between the seats

They ask	Senior full-stack answer	Technical architect answer
Microservices?	How I would call the other service.	The business capability, the data owner, the team that deploys it, the failure the user sees, and why a module is enough until those are real.
How should the Angular app authenticate?	An interceptor and a token in storage.	Who the users are, where the token lives, what a stolen script can do, and whether a backend for frontend removes the token from the browser.
The page is slow.	A missing index or a heavy bundle.	The budget, which percentile, which request is on the user path, and which delay we moved to a message on purpose.
Can we rewrite the legacy app?	The target framework.	The slice that earns the rewrite, the people who query the old database, the rollback, and the date the old path dies.
Which Azure service?	The service I used last year.	The operational load, the scale assumption, the identity, and the pillar it stresses: reliability, security, cost, operations, or performance.

Answer shape that scores as an architect

1. First sentence: the decision, not the background.
2. Second beat: the constraint that made it (team size, data ownership, latency, threat, cost driver).
3. Third beat: the option you rejected and the pain it would have added.
4. Fourth beat: how you would know the decision is failing. A metric, an incident, or a date.
5. Stop. If they lean in, give one story from your eight years. If you do not know a limit, name the document or the bill you would open. Invented numbers lose the room.

Score each artifact from 0 to 2

- Decision: a reader can see what you chose.
- Tradeoff: you named what you gave up.
- Operable: a person on call can see how it fails and how it rolls back.
- Evidence: a scenario, a diagram, or a number you labeled as an assumption or a measurement.
- Six points out of eight is a hire signal on that artifact. A correct diagram with no rejected option still sounds like a senior developer.

The system you will design all month

Harbor is a B2B wholesale portal. Buyer companies place orders. Internal staff release credit holds and watch fulfillment exceptions. You will not build Harbor as a product this month. You will make the architecture decisions a team of about eight to twelve engineers could build.

Label these numbers as assumptions every time you say them. If a panel changes a number, change the design. Peak is about 40 order submits a second. About 2,000 buyer companies and 15,000 users. The accept path, meaning the order is durable before the API returns, targets p95 under 400 milliseconds. Email and other notices can take a minute. One Azure region until an SLO forces a second region. Card numbers stay at the payment provider. You do not store them. The team is two squads that can still talk every day. A million events a second is a different system. Do not design that system by accident.

The containers you are allowed to use

- Buyer portal and ops portal: Angular. They may be two apps or two areas. Day 19 forces the choice.
- A backend for frontend, the BFF: ASP.NET Core. The browser's only API.
- Harbor application: a modular monolith on ASP.NET Core with modules for Catalog, Ordering, and Billing, until an ADR splits one out.
- Azure SQL: one database, schema per module, unless a later decision says otherwise.
- Azure Service Bus: something that must survive a process restart and may be handled twice.
- A notification worker: Azure Functions, isolated worker, because mail is a different failure domain from the accept path.
- Microsoft Entra ID for staff. Microsoft Entra External ID for buyers on a new system. Azure AD B2C only if you are extending an estate that already has it.
- Key Vault and managed identity. No passwords in appsettings, and nothing secret in the Angular environment files. Those files ship to the browser.

Repo to create on day 1

Create a private git repository named harbor-architecture. Folders: docs/principles, docs/c4, docs/adr, docs/quality, docs/security, docs/reviews. Write markdown. Diagrams can be Mermaid or a photo of a page. This repository is the portfolio. Do not put real customer data, secrets, or your employer's private designs in it.

How to use AI for 30 days without handing it the decision

AI is a critic and a drafter in this plan. It is not the architect. Cursor, in this repository, is the right tool because it can see your ADR next to your principles. A chat window is the right tool for a mock panel. Copilot is fine for a code sketch inside a depth spike. The failure mode is a fluent document you cannot redraw the next morning.

Rules you do not break

1. Paste the constraints before the question: team size, scale assumption, one region, no card data, .NET and Angular, existing database if any.
2. Ask for the strongest objection and the missing quality attribute. Do not ask 'is this a good architecture'.
3. You write the decision sentence yourself. The model may propose options you then accept or reject.
4. Throw away invented prices, quotas, latency numbers, and legal claims. If a number matters, label it assumption or name the calculator, the bill, or the official limit.
5. Do not paste secrets, production connection strings, personal data, or unreleased financials. Use a company-approved tenant when the material is work material.
6. Redraw from memory after every AI session. If you cannot, the model owns the design and you do not.
7. Check any large sample for a license you do not have. Prefer your own Harbor names over a pasted textbook diagram.

Session prompt, paste at the start of every AI block

```
You are a staff-level reviewer of a .NET and Angular system called Harbor, a B2B wholesale portal.
Constraints: about 8 to 12 engineers, about 40 order submits per second at peak (assumption), one
Azure region, Azure SQL, no card data stored, buyers on Entra External ID, staff on Entra ID,
Angular portals, ASP.NET Core on .NET 10.
Your job is to attack my draft. Do not rewrite it unless I ask.
Reply in this order:
1. The decision you think I actually made, in one sentence.
2. The strongest objection.
3. The quality attribute I ignored (name the stimulus and the missing measure).
4. What fails at 2 a.m., and whether rollback is real.
5. One option I did not consider, and when it would be better.
Do not invent Azure prices, quotas, or benchmark numbers. If a limit matters, name the document I
should open.
Mark any sentence I should not say out loud because it is vague.
```

Each day adds a shorter prompt aimed at that day's artifact. Run the session prompt first, then the day's prompt, on the text you wrote yourself. If you let the model write the first draft, you must delete it and write the decision sentence again before you score the day.

Where Cursor fits

- Keep principles.md open. Ask Cursor to list sentences in the ADR that contradict a principle. You decide which side to change.
- Ask for a Mermaid container diagram from your own container list. Then draw it on paper without looking.
- Ask for a hostile review of a pull request that breaks module boundaries. That is the tech-lead spike, still useful on days 18 and 20.
- Do not ask Cursor to 'design Harbor properly' and then commit the answer. That produces a portfolio you cannot defend.

What the model must not decide

Whether you promise 99.99 percent. Whether you adopt Kubernetes. Whether you are in scope for a regulation. Whether a rewrite is funded. Whether a person on your team is ready to own a service. Those are your decisions, made with people who carry the cost.

Materials

This document is the primary study material. External sources are there so you can see the original wording of an idea. You do not buy a course. You do not watch a 40-hour video series during these 30 days.

Have these open

- .NET 10 SDK, so any sketch you write still compiles.
- Current Node LTS and the Angular CLI major you have decided to speak about.
- A markdown editor. The repository above.
- Paper. The interview and the review still happen on a whiteboard.
- Optional: Docker if you want .NET Aspire locally in week 3. Not required to finish the 30 days.
- An Azure subscription is optional. Draw the hosts. Do not spend the month in the portal.

Books, used as references, not as a reading challenge

Source	Use it for	Time you should spend
Fundamentals of Software Architecture, Richards and Ford	Characteristics, decisions, and what not to diagram	About 4 hours, days 1 to 4
Software Architecture: The Hard Parts, Ford, Richards, Sadalage, Dehghani	Data ownership, granularity, sagas	About 4 hours, days 9 and 10
Designing Data-Intensive Applications, Kleppmann	Chapters 1, 7, and 8 only: reliability, transactions, distributed-system trouble	About 5 hours, days 10 and 15
Team Topologies, Skelton and Pais	Four team types and three interaction modes	About 2 hours, day 5
Building Evolutionary Architectures, Ford, Parsons, Kua	Fitness functions	About 2 hours, day 18
Release It!, Nygard, second edition	Timeouts, bulkheads, and stability patterns	About 3 hours, day 15
Accelerate, Forsgren, Humble, Kim	The four delivery metrics, as health of the system not as a slogan	About 90 minutes, day 20
Site Reliability Engineering, free at sre.google	Embracing Risk, and Service Level Objectives	About 2 hours, day 15

Free primary sources

- Azure Well-Architected Framework and the Azure Architecture Center, on Microsoft Learn. Pillars and the application-architecture decision guides.
- Microsoft identity platform documentation: authorization code flow with PKCE, and the on-behalf-of flow so you know when not to use it.
- ASP.NET Core security and OpenAPI documentation. Problem Details. Cookie and OpenID Connect handlers.
- .NET Aspire overview, so you can say what it does locally and what it does not decide in production.
- angular.dev: signals, standalone components, lazy routes, and the releases page for the support window.
- c4model.com for diagram levels. adr.github.io and Michael Nygard's original ADR post for the decision format.
- arc42, the short template, as a checklist when a document feels bloated. You do not fill all twelve sections for Harbor.
- OWASP API Security Top 10, current edition. Especially broken object level authorization.
- Google's SRE book chapters named above. ISO/IEC 25010 only as a checklist of quality names, not as the way you speak.

If a book is not on your shelf, the explanation in this document is enough to do the day's artifact. The book makes you harder to shake in a follow-up question. Do not stall the calendar waiting for a delivery.

Day 1 — The seat you are taking

Outcome: you can say what a technical architect owns, what you leave with the tech lead, and who else must agree before a decision is real. You write your positioning line in your own words and a stakeholder map for Harbor.

Time box (4 hours)

- 70 min: this day, including the comparison table, out loud.
- 30 min: Fundamentals of Software Architecture, the chapters on the architect's role and expectations. Stop at 30 minutes.
- 70 min: positioning line plus the Harbor stakeholder map.
- 30 min: speak. 30 min: AI critic. 10 min: rewrite the positioning line from memory.

Study material

A senior full-stack developer owns a change through code, tests, and a pull request. A tech lead owns that plus the team's plan: review quality, the release, mentoring, and the design of the service the team ships. A technical architect owns decisions that more than one team will live with for a year or more: where the boundaries are, who owns the data, how identity works, which qualities you will actually pay for, and which standards are written down. You still read code. You stop being the fastest person to take every story.

A solution architect, especially in a services firm, spends more of the week in a workshop and a written proposal: scope, integrations, assumptions, risks, and a shape the client can fund. The engineering underneath is the same. If you cannot explain the failure mode, you are writing a brochure. An enterprise architect sets principles across many products and rarely designs one accept path. That is a later career, not day 30.

Decision rights are the part people skip. You recommend. You do not secretly own the budget, the residual security risk, or the business priority. For Harbor, a typical split is: you recommend modular monolith over microservices; the engineering manager accepts the staffing consequence; security accepts the residual risk of one database; the product owner accepts that email is not in the 400 millisecond budget. Write the names of the roles even if you do not yet have the people. A decision with no owner is a slide.

Influence without authority is the job on the weeks you do not manage the team. You earn it with a short document, a real alternative, and time in the other team's problem. You lose it by announcing a target architecture in week one, by reviewing style instead of boundaries, and by using AI output you cannot explain.

Situation	Tech lead does	Technical architect does
A module wants a new NuGet package	Reviews the license and the call site in the PR	Writes nothing, unless the package sets a platform standard
Two squads both want to write orders	Runs the design conversation	Names the single writer and records the decision
Production is down	Leads the incident for their service	Joins if the boundary or the data owner is the cause. Does not take the keyboard by rank.
A director asks for a rewrite	Estimates the team's slice	Says which slice, which risk, and what will not be rewritten

Stakeholder map for Harbor

List eight roles and one sentence each: what they need from the architecture, and what they can veto. Buyer admin, warehouse lead, product owner, two squad leads, security, the person who pays the Azure bill, and the reporting analyst who already runs SQL against something. That last person is how modernization projects die. Put them on the map on day 1, not as a surprise on day 23.

Say this

I recommend the shape and I record the tradeoff. The budget owner accepts cost, security accepts residual risk, and the tech lead owns the team's release. If I cannot name those people, the decision is not made yet.

AI session

```
Here is my positioning line and my Harbor stakeholder map.
Attack them.
Tell me where I still sound like the person who will implement every story myself.
Tell me which stakeholder can veto a decision, and which line I wrote that pretends they cannot.
Do not improve my tone. Improve the decision rights.
```

Evening artifact

1. Positioning line, six lines or fewer, in docs/reviews/positioning.md.
2. Stakeholder map with vetoes.
3. Three sentences: work you will stop doing in the first month of the role so the tech leads still own delivery.

Q1. What is the difference between a tech lead and a technical architect?

Answer. The tech lead owns one team's delivery: design of their service, review, release, and people. The technical architect owns decisions that cross teams or last long enough to be expensive to undo: boundaries, data ownership, identity, quality attributes, and written standards. I still read code. I do not take every story.

Say this as an architect

On Harbor I would let the squad lead own the Orders module release. I would own the decision that Ordering is the only writer of an order, and I would write that down.

Day 2 — Quality attributes you can test

Outcome: you can turn 'fast, secure, and scalable' into three scenarios a team could accept or reject. You can say what you are giving up.

Time box (4 hours)

- 70 min: scenarios and tradeoffs below.
- 30 min: Richards and Ford on architecture characteristics. Or Microsoft's Well-Architected overview if the book is not here.
- 70 min: write three Harbor scenarios in the template.
- 30 min speak, 30 min AI, 10 min redraw one scenario from memory.

Study material

A quality attribute is a property the system must have, expressed so you can argue about it. 'Scalable' is not one. A scenario is. The form used at the Software Engineering Institute is: source, stimulus, artifact, environment, response, response measure. You do not need the jargon in the room. You do need all six parts on the page.

Source: who or what starts it. Stimulus: the event. Artifact: the part of the system that must respond. Environment: normal load, peak, or already failing. Response: what the system does. Response measure: a number, a window, and how you would observe it.

Harbor accept-path example. Source: a buyer user. Stimulus: submits an order. Artifact: the Ordering module. Environment: peak, about 40 submits a second, assumption. Response: the order and its outbox row are committed, and the API returns the order id. Response measure: p95 under 400 milliseconds over a calendar month, excluding the payment provider's time if payment is asynchronous. If you include the provider, say so, and do not promise 400 milliseconds for their outage.

Percentiles matter. An average of 200 milliseconds can hide a p95 of two seconds. Availability is a different promise from latency. 99.9 percent of a 30-day month is about 43 minutes of unavailability. 99.99 percent is about 4 minutes. Do not offer 99.99 percent for Harbor in one region with one database. You have not paid for it.

Tradeoffs you should be able to say without a slide. Consistency against availability: if Azure SQL is down, Harbor rejects the submit. You do not accept orders you cannot store. Latency against freshness: the catalog page may be a few minutes old; the stock check on submit may not. Coupling against duplication: a read model copies order status into a search store; that copy can be stale. Cost against resilience: a second region doubles a class of bill and forces a data story you do not have yet. Speed of change against uniformity: two squads in one deployable wait for each other sometimes; two deployables without a platform team wait for incidents instead.

A fitness function is an automated check that an architectural choice still holds. Examples you can actually build later this month: a test that the Catalog project does not reference Ordering infrastructure; a unique index on the idempotency key; an accessibility scan on the order route; a pipeline check that fails a new public HTTP client with no timeout. A fitness function without a decision is just a test. Day 18 comes back to this. Today, name two you would want for Harbor.

ISO/IEC 25010 lists qualities such as reliability, security, performance efficiency, maintainability, and compatibility. Use it as a checklist so you notice the one you ignored. Do not recite the standard in the interview. Speak the scenario.

Template

```
Quality:
Source:
Stimulus:
Artifact:
Environment:
Response:
Response measure (number, window, how observed):
Assumption or measurement:
What we give up to get this:
Revisit when:
```

Say this

Fast is not a requirement. For Harbor, the buyer submit at peak commits the order and returns in under 400 milliseconds at p95. That number is an assumption until we measure it. Mail is outside that budget. If SQL is down we fail the submit, because we will not take an order we cannot store.

Interview trap

Listing twelve qualities with no priority. An architect ranks them for this system. Harbor ranks integrity of the order and tenant isolation above a beautiful catalog animation. Say the rank.

AI session

```
Here are three quality attribute scenarios for Harbor.
For each one, tell me if the response measure can be observed, or if I used a vague word.
Point at the measure that fights another measure.
Do not add new Azure services. Do not invent a benchmark.
```

Evening artifact

1. Scenario: latency of order submit.
2. Scenario: a buyer must not read another company's order.
3. Scenario: a Catalog change ships without an Ordering code change. Say what 'without' means for a modular monolith (no Ordering pull request, not a separate deployable).

Q1. How do you handle non-functional requirements?

Answer. I write them as scenarios: who triggers what, against which part of the system, under which conditions, with a response and a measure. I rank them for this product. I label numbers as assumptions until we have a measurement. I name the quality we are spending less on so the tradeoff is visible.

Say this as an architect

On Harbor, durability of the order beats a second region. I would rather fail a submit than accept an order we cannot store.

Day 3 — Draw the system at the right altitude

Outcome: you can draw Harbor's context and containers in one sitting, and you can say which diagram you refused to draw.

Time box (4 hours)

- 60 min: C4 levels below.
- 30 min: c4model.com, the levels page only.
- 90 min: draw context and containers on paper, then write them in markdown.
- 30 min speak, 30 min AI, 10 min redraw containers with the laptop closed.

Study material

The C4 model, from Simon Brown, is four levels of zoom. Context: Harbor and the people and systems outside it. Container: the applications and data stores that run, in the Docker sense of a container only by coincidence. A container here is the Angular app, the BFF, the database, the bus. Component: the major parts inside one container, drawn only when you are discussing that container. Code: classes, which do not belong in an architecture review unless someone asks how a specific module is shaped.

A diagram that does not change a decision is decoration. Every box needs a responsibility you can say in one sentence. Every arrow needs a protocol or a 'why'. If two boxes always deploy together and fail together, they are probably one box at container level.

Harbor context, in sentences. Buyers use Harbor to place wholesale orders. Ops staff use Harbor to manage credit and exceptions. Harbor uses Microsoft Entra to authenticate both, in two different directories or configurations. Harbor uses a payment provider to authorize payment and does not store card numbers. Harbor sends mail through an email provider. A reporting analyst is a person, not a system, until they have a warehouse. If they query SQL directly, draw that arrow and label it 'legacy, painful, explicit'.

Harbor containers, the default you will attack on later days. Buyer Angular app. Ops Angular app, or a second area if day 19 says so. BFF, ASP.NET Core, the only thing the browser calls. Harbor API, the modular monolith. Azure SQL. Azure Service Bus. Notification worker on Functions. Key Vault. The telemetry sink, named as Azure Monitor or Application Insights, not as 'logging'. Entra sits outside, already on the context diagram. Do not draw a Kubernetes cluster. You have not earned it.

```
Buyer ---- HTTPS ----> Buyer Angular ---- HTTPS/cookie ----> BFF
Ops user ---- HTTPS ----> Ops Angular ---- HTTPS/cookie ----> BFF
BFF ---- HTTPS + app identity ----> Harbor modular monolith
Harbor monolith ---- TDS/managed identity ----> Azure SQL
Harbor monolith ---- Service Bus ----> Notification worker ----> Email provider
Harbor monolith ---- HTTPS ----> Payment provider
Buyer and ops ---- OIDC ----> Entra (two audiences)
```

arc42 is a fuller template: goals, constraints, context, solution strategy, building blocks, runtime view, deployment view, crosscutting concepts, decisions, quality, risks, glossary. For Harbor you will fill a short form of goals, constraints, context, building blocks, a runtime view of submit order, decisions, quality scenarios, and risks. You will not write a 40-page arc42 this month. Bloated documentation is a risk. So is a diagram with no decisions next to it.

Say this

Context shows who is outside Harbor and what we refuse to store, which includes card numbers. Containers show the Angular apps, the BFF, one modular monolith, SQL, Service Bus, and a worker. I will draw components only for the accept path, and I will not draw classes in a review.

AI session

```
Here is my Harbor context list and container list.
Which box has no responsibility I could say out loud?
Which arrow has no reason to be synchronous?
Where did I draw a technology instead of a container responsibility?
Do not add services I did not list.
```

Evening artifact

1. docs/c4/context.md with actors, external systems, and one sentence on what Harbor will not do.
2. docs/c4/containers.md with the boxes and arrows above, edited to match your wording.
3. A note of the first component diagram you will need: submit order only.

Day 4 — Write a decision someone can revisit

Outcome: you can write an architecture decision record that a squad lead can follow without you in the room. You finish ADR-001: Harbor stays a modular monolith for this stage.

Time box (4 hours)

- 50 min: the ADR rules below.
- 30 min: Nygard's ADR article or adr.github.io, the template only.
- 100 min: write ADR-001 yourself. The sample further down is a reference, not your file. Write yours first, then compare.
- 30 min speak, 30 min AI, 10 min restate the decision and the revisit trigger with the page turned over.

Study material

An architecture decision record is a short document with a status, a context, a decision, and consequences. Michael Nygard described that shape. The point is not the headings. The point is that the next person can see what you chose, what you refused, and what would make the choice obsolete. A wiki page titled 'Architecture' that is edited forever is not a decision. A decision has a date and a status: proposed, accepted, superseded.

Write options as real alternatives, including the one you reject. 'Microservices versus monolith' is too coarse. For Harbor the options are: one modular monolith with module boundaries enforced in code; a handful of independently deployed services on day one; a monolith with no internal boundaries. The third option is how you get a rewrite in two years. The second option is how a team of ten spends the half on pipelines, distributed debugging, and data that no longer shares a transaction.

Consequences are both good and bad. Good: one deployable, one database transaction for an order and its outbox, simpler local development. Bad: a faulty Catalog change can still take down Ordering if you share a process. You mitigate that with module boundaries, tests, and a separate worker for mail, not with a speech about discipline. Also bad: you cannot scale the catalog read path by giving it its own replica count. You accept that until measurement says the accept path or the catalog is the saturated part.

A revisit trigger is mandatory. Examples: a module must be owned by a team that cannot share the release train; a measured hotspot needs independent scale; a compliance boundary forbids shared hosting; the process fails its SLO because of a neighbor module and isolation inside the process is not enough. 'When we feel ready' is not a trigger.

Reference ADR, read only after you draft yours

```
ADR-001: Harbor ships as a modular monolith
Status: Accepted
Date: (today)

Context:
```

```
Two squads, about 8 to 12 engineers, one product, one Azure region.
Peak submit is assumed at about 40 requests a second.
Ordering and the outbox row must commit together.
We have no platform team that runs Kubernetes.

Decision:
One ASP.NET Core deployable with modules Catalog, Ordering, and Billing.
Each module owns its schema. Code may not query another module's tables.
Mail leaves the process through Service Bus to a Functions worker.
We do not create independently deployed business services yet.

Options rejected:
Separate services for Catalog, Ordering, and Billing now.
A single project with no module boundaries.

Consequences:
We keep a local transaction for the order and the outbox.
A bad deploy can still couple the modules at runtime.
Catalog cannot scale on its own until we split it on purpose.

Revisit when:
A squad must release on a different cadence, or a measured resource limit sits inside one module
and cannot be fixed in-process.
```

Say this

I chose a modular monolith for Harbor because the team is still one release conversation and the order needs a local transaction. I rejected microservices until a module has a different cadence, a different scale problem we measured, or a compliance boundary. The cost is that one bad deploy can affect the whole API. The worker for mail is already separate because it fails differently.

Interview trap

Calling it a monolith as an apology, or calling it microservices because there are three class libraries. Deployment independence and data ownership are the test. A class library is not a service.

AI session

```
Review ADR-001, which I wrote myself.
Quote the revisit trigger. If it is vague, say so.
Name a consequence I treated as only positive.
Give me the strongest case for splitting Ordering out now, then say whether my constraints defeat
that case.
Do not rewrite the ADR.
```

Evening artifact

1. docs/adr/001-modular-monolith.md in your words.
2. Status, context, decision, options rejected, consequences, revisit trigger.
3. One sentence a director can read, copied into the top of the file.

Q1. Monolith or microservices?

Answer. For Harbor, a modular monolith. The team is small enough to share a release, and the order plus the outbox need one transaction. Microservices would add network failures and data split before we have a platform team or a measured hotspot. I would split a module out when its cadence, scale, or compliance boundary is actually different. Mail is already a separate worker because its failure domain is different.

Say this as an architect

I would enforce the modules with a test that forbids cross-schema reads, not with a meeting.

Day 5 — Boundaries, language, and the teams who own them

Outcome: you can name Harbor's bounded contexts, who is allowed to write each piece of data, and which team shape matches ADR-001.

Time box (4 hours)

- 70 min: contexts and team types below.
- 40 min: Team Topologies, the four team types and the interaction modes. Skip the rest today.
- 70 min: a one-page context map.
- 30 min speak, 30 min AI, 10 min redraw the map.

Study material

A bounded context is a boundary inside which a word means one thing and a model is consistent. In Harbor, 'order' in Ordering means the commercial commitment: buyer, lines, price as copied at submit time, status. 'Order' in the warehouse's language might mean a pick list. If you use one class for both, you will leak warehouse fields into the buyer API. Catalog owns products and list prices. Ordering owns the order, copying the price at submit so a later catalog edit does not rewrite history. Billing owns invoices and credit. Notification owns delivery of messages, not the business fact that an order exists. Identity owns who the user is; it does not own what the user may do to an order. That authorization rule lives next to the order.

A context map describes the relationship between contexts. Shared kernel: a tiny model both sides change together. Dangerous if it grows. Customer-supplier: Ordering needs something from Catalog and can ask for a change. Conformist: you take an external model as-is, which is how you should treat the payment provider. Anti-corruption layer: a translation at the edge so the provider's names never enter your domain. Harbor's payment adapter is that layer. Published language: the message schema on Service Bus.

Conway's law is the observation that the system copies the communication structure of the organization. Use it as a design input, not a joke. If you create four services and one team, you have given one team four operational surfaces. If you create two squads and forbid them to speak, a modular monolith will still rot at the boundary. Team Topologies gives you four team types. Stream-aligned teams own a flow of work, here the buyer journey and the ops journey, or one team per journey if you split later. A platform team provides a paved path: pipeline, observability, a service template. An enabling team teaches a skill, such as threat modeling, and then leaves. A complicated-subsystem team owns a rare specialty. Harbor does not have that problem yet. Do not invent a platform department for twelve people. A thin shared pipeline owned by a rotating pair is the honest platform.

Interaction modes: collaboration is two teams working together for a defined period, expensive and sometimes necessary at a new boundary. X-as-a-service is a clear API with a small waiting time. Facilitating is the enabling team. Harbor's two squads collaborate on the monolith's boundary rules, then treat modules as an internal contract. They are not two companies.

Cognitive load is the reason you still draw boundaries inside one deployable. A person should be able to change Catalog without holding the Ordering state machine in their head. The Angular app mirrors this. Feature areas or libraries follow the same words: catalog, ordering, billing. A shared UI kit is not a bounded context. It is a dependency with a different rule: visual consistency, no business decisions inside a button.

Say this

Catalog owns the product and the list price. Ordering copies the price onto the order at submit and is the only writer of order status. The payment provider's model stops at an anti-corruption layer. Two squads share one deployable until a revisit trigger in ADR-001 fires. I will not create a platform team to look mature.

AI session

```
Here is my Harbor context map: contexts, who writes which data, and the team shape.
Find a word I used to mean two things.
Find data with two writers.
Tell me if my team shape contradicts ADR-001.
Do not propose a new vendor.
```

Evening artifact

1. One page: Catalog, Ordering, Billing, Notification, Identity. Writer, data, and what is copied rather than shared.
2. Mark the payment adapter as anti-corruption.
3. One paragraph on team shape that matches ADR-001.

Day 6 — The shape of Angular in front of .NET

Outcome: you can walk the path from a button click to a committed order, and you can say where the token is allowed to live.

Time box (4 hours)

- 80 min: BFF, cookies, and PKCE below. Sketch the sequence on paper.
- 30 min: Microsoft identity docs, authorization code flow with PKCE, skim only.
- 70 min: write the runtime view of submit order in docs/c4/submit-order.md.
- 30 min speak, 30 min AI, 10 min redraw the sequence.

Study material

The browser is an untrusted client. Anything you ship in JavaScript can be read by a script that should not be there, and anything the buyer posts can be rewritten. Prices, roles, and tenant ids in the request body are hints at most. The server recomputes the price from Catalog data, takes the tenant from the authenticated session, and ignores a client-supplied role.

For a first-party Angular app, the default architecture in this plan is a backend for frontend. The Angular app calls only the BFF, same site. The BFF runs the OpenID Connect authorization code flow with PKCE against Entra, keeps the tokens on the server, and gives the browser an HttpOnly, Secure, SameSite cookie. The Angular app does not see the access token, so it cannot put one in localStorage. That removes a common theft path. The cost is a BFF you must operate, and you must handle CSRF because the browser will send the cookie.

CSRF: SameSite=Lax or Strict stops many cross-site posts. For cookie-authenticated writes, also use an anti-forgery token. CORS is not your authorization model. CORS decides which browser origins may read a response. A non-browser client ignores it. Do not describe CORS as 'locking the API'.

The pure SPA alternative is still valid: authorization code plus PKCE in the browser, tokens in memory, refresh done carefully, nothing in localStorage. Use it when a BFF is truly unjustified, for example a temporary app or a host that cannot run the cookie site. Say the threat you accepted: any XSS can act as the user while the token is in memory, and a mistake in storage becomes a lasting theft. The BFF moves that bar to the cookie and the BFF's own session store.

Downstream, prefer the BFF or the monolith to reach Azure SQL and Service Bus with managed identity. Do not forward the user's raw token to every internal API by habit. The on-behalf-of flow exists when a downstream API must independently see the user and enforce its own delegated permission. It is more moving parts. For Harbor, the monolith enforces authorization itself using the user id, the tenant, and the roles the BFF established. Internal calls use the app's identity. Revisit on-behalf-of when a downstream team refuses to trust the monolith.

Buyer identity and staff identity are different trust zones. Staff live in the workforce tenant. Buyers, for a new build, live in Entra External ID. Do not mix them in one Angular app with a boolean. Two app registrations, two audiences, and the ops app is not deployed to the public buyer host if you can avoid it. A buyer token must fail on ops endpoints even if someone copies the URL.

The BFF is not a place to hide business rules. It authenticates, applies a few client-specific shapes, and calls the monolith. If price calculation starts living in the BFF, you have two Ordering models. Aggregation of a read screen is fair. A decision about whether an order is valid is not.

```
Sequence: submit order
1. Angular POSTs /api/orders with the cookie and an Idempotency-Key.
   Body has sku and quantity. It does not have a price the server trusts.
2. BFF authenticates the cookie, rejects a missing anti-forgery token, calls Harbor.
3. Ordering loads the buyer tenant from the session, loads prices from Catalog's public method or a
copied price list, rejects a tampered tenant.
4. Ordering inserts the order and the outbox row in one SQL transaction.
5. API returns 201 and the order id.
6. A relay publishes the outbox row to Service Bus.
7. The worker sends mail. The user is not waiting on step 7.
```

A thin code sketch is enough if you want your hands in the tools today. Cookie authentication plus the OpenID Connect handler in ASP.NET Core, and an Angular app that calls a relative /api URL with credentials included and has no token interceptor. If the sketch grows past an hour, stop. The sequence above is the day's real output.

Say this

The Angular app talks only to a BFF. Tokens stay on the server. The browser gets an HttpOnly cookie, and writes carry an anti-forgery token. The server ignores any price or tenant id in the body. Mail is after the commit. I would put tokens in the SPA only if I had rejected the BFF on purpose and accepted the XSS exposure.

AI session

```
Here is my submit-order sequence.
Where could a malicious buyer change the price or the tenant?
Where is the token, exactly?
What happens if the process dies after the SQL commit and before the message is published?
Do not add an API gateway unless you can show why the BFF is not enough.
```

Evening artifact

1. docs/c4/submit-order.md, the sequence in your words.
2. A short note: token location, cookie flags, what the client is not allowed to decide.
3. One rejected option: access token in localStorage, and the sentence that says why.

Q1. Where do you store the token in an Angular app?

Answer. For Harbor I do not store it in the Angular app. The BFF holds the tokens and the browser holds an HttpOnly Secure cookie. localStorage is readable by any script on the page, so I will not put a bearer token there. If we had no BFF, I would keep the token in memory and say out loud that XSS can still act as the user until the page closes.

> **Say this as an architect**
> Authorization still happens on the server. The UI hiding a button is not a control.

Day 7 — Principles, then a hostile read

Outcome: a one-page principles document that ADR-001 and the submit path already obey. You can spot a sentence that is only a slogan.

Time box (4 hours)

- 40 min: read the principle rules below.
- 80 min: write eight principles or fewer, each with a consequence.
- 40 min: check ADR-001 and the submit sequence against them. Fix either the principle or the decision.
- 40 min: speak a five-minute briefing: who Harbor is, the decision, the quality you ranked first.
- 30 min: AI critic on the principles. 10 min: recite them without the page.

Study material

A principle is a rule you are willing to enforce. 'We build quality in' is not a principle. 'The browser is untrusted; prices and tenant ids are taken from the server session and the database' is a principle, because a pull request can violate it and a reviewer can point at the sentence. Each principle needs a consequence that can hurt. If there is no case where the principle makes you slower, it is a poster.

Start from these eight, then rewrite them until they sound like you. One owner per data set. The browser is an untrusted client. A synchronous call has a timeout. A message may be delivered more than once, so the consumer is idempotent. Prefer a module boundary before a network boundary. An accepted decision names a revisit trigger. We measure a promise we make about latency or availability. Standards are written, and an exception is an ADR, not a hallway agreement.

Add a non-principle list so you do not pretend taste is architecture. Formatting, folder names inside a feature, and whether a component is signals or a local observable are team conventions. They can be written. They are not why you were hired as an architect, unless they break a boundary.

The week review is a spoken briefing, five to ten minutes. Shape: who the users are; the container picture; ADR-001 in four sentences; the accept-path measure; the threat you designed against; what would make you reopen the decision. Record yourself on the phone. If you hear 'basically' and 'just', cut those sentences and say the tradeoff again.

Say this

Our principles are short because we intend to enforce them. The one I will argue hardest this month is a single writer per data set, and the one that will slow us down is the timeout on every synchronous call, including the ones that 'usually return quickly'.

AI session

```
Here are my principles, ADR-001, and the submit-order sequence.
List contradictions.
List principles that cannot be violated in a pull request because they are too vague to enforce.
Do not add principles to make the list look complete.
```

Evening artifact

1. docs/principles.md, one page.
2. A five-minute recording, deleted after you note three sentences you will change.
3. A score of the week using the front-matter rubric. Write the weakest artifact. Day 29 will thank you.

Day 8 — How Harbor talks to the rest of the world

Outcome: for each Harbor flow you can say whether it is a method call, an HTTP call, or a message, and what the user sees when the other side is slow or down.

Time box (4 hours)

- 70 min: integration styles and the timeout budget below.
- 30 min: Azure Architecture Center page on communication in a microservices or distributed system, the styles section only. Or Hohpe's Enterprise Integration Patterns overview if you already know the Microsoft page.
- 80 min: a table of Harbor flows: sync, async, or in-process, with the failure the user sees.
- 30 min speak, 30 min AI, 10 min redraw the table.

Study material

There are four classic ways two parts of a business exchange data. A shared database, which couples them forever and is the legacy you will meet on day 23. A file drop, which is still how some warehouses work and is acceptable when the latency is hours and the contract is a documented file. A remote call, HTTP or gRPC, where the caller waits. A message, where the caller records the fact and someone else reacts. Pick the style from who is waiting and who owns the data, not from a preference for events.

Inside the Harbor modular monolith, Catalog and Ordering talking in-process is a method call, not a network call. Do not put HTTP between two modules that deploy together just so the diagram looks like services. That pays for failure modes you do not get independence in return for. The method is still a boundary: Ordering calls a public method on Catalog, or reads a price list Catalog published into Ordering's schema on purpose. It does not join Catalog tables.

Use a synchronous call when a person is waiting and you need the answer to finish their action. Harbor's payment authorization is the example, and only if the product insists the buyer sees 'paid' before the screen returns. Then the payment call has a timeout that is part of the architecture, not a default of 100 seconds buried in HttpClient. If the provider is slower than the page should be, change the product shape: accept the order as payment pending, return 201, and let a worker finish the authorization. Do not hold an ASP.NET request open to look reliable.

Use a message when the work can finish after the user has a response, or when the receiver is down and the fact must not be lost. Mail, search indexing, and a warehouse pick list are messages. Azure Service Bus is the broker in this plan because you want a durable queue, a dead-letter queue, and competing consumers. A message is at least once. Duplicate detection on Service Bus is a time window, not a business guarantee that the mail sends once. The consumer must be safe to run twice.

A contract is part of the architecture. For HTTP, OpenAPI is the contract, checked so a breaking change fails CI. For messages, a schema with a version field is the contract. Add fields. Do not change the meaning of an

existing field. Do not use a topic as a shared database that every service queries by replaying from the dawn of time unless you have actually designed for that. Harbor does not need event sourcing to ship.

Timeouts compose. If the user-facing budget is 400 milliseconds at p95, the calls inside that request must be shorter, and you need a number for each. An assumption you may write down: catalog price lookup inside the process, a few milliseconds; SQL commit, the bulk of the budget; payment, not on this request if you chose pending. Say the assumption. A timeout with no retry policy is unfinished, and a retry policy on a non-idempotent payment call is a double charge. Day 15 goes deeper. Today, name the timeout owner.

Flow	Style	User impact if the other side is down
Buyer loads a product	Read, in-process or a replica	Catalog page fails or shows a stale snapshot you labeled
Buyer submits an order	Local transaction in Ordering	Submit fails. No half-saved order.
Payment authorization, if on the request	Sync HTTPS with a short timeout	Order stays pending or the submit fails. Never a silent double charge.
Confirmation mail	Outbox then Service Bus	Order is already accepted. Mail is late. A dead-letter is an alert, not a lost order.
Warehouse pick	Message or a file, their contract	Order is accepted. Pick is delayed. Their outage is visible as a status, not as an HTTP 500 to the buyer.

Say this

I keep Catalog and Ordering in-process until they deploy separately. I use HTTP when the user is waiting and I own the timeout. I use Service Bus after the order is durable, for mail and for partners who must not be on the buyer's request. The broker can deliver twice. The handler is idempotent. I do not share the database with the warehouse as the integration.

Interview trap

Saying 'we are event-driven' and then describing a chain of synchronous calls with a bus in the middle. A bus does not remove coupling if the user still waits for every consumer.

AI session

```
Here is my Harbor integration table.
Which synchronous call will blow the 400 millisecond assumption if the dependency stalls?
Which message did I use where the user actually needs the answer on the request?
Where did I imply exactly-once delivery?
Do not add Kafka unless you can say why Service Bus fails the requirement.
```

Evening artifact

1. docs/c4/integrations.md with the table in your words.

2. One sentence on the payment choice: on the request, or pending. Pick one and name the product consequence.
3. The timeout you would set on that payment call, labeled as an assumption.

Q1. REST or messaging?

Answer. Both, for different waiting times. Submit commits in SQL and returns. Mail and the warehouse are messages because the buyer should not wait on them, and those systems fail independently. I would not put HTTP between Catalog and Ordering while they are the same deployable. I would not claim the bus gives me exactly-once delivery.

> **Say this as an architect**
>
> If payment must be known before the button finishes, that call is synchronous, it has a timeout, and it is not blindly retried.

Day 9 — Who owns the data

Outcome: you can point at every important table family and name one writer. You can explain the outbox and a tenant filter without drawing a second database you do not need.

Time box (4 hours)

- 70 min: ownership, outbox, and tenancy below.
- 40 min: Software Architecture: The Hard Parts, the data-ownership chapters, or this document again if the book is not here.
- 70 min: schema sketch and the outbox transaction in words.
- 30 min speak, 30 min AI, 10 min redraw who writes an order.

Study material

Data ownership is the architecture. Service boundaries that do not match write ownership are fiction. In Harbor one Azure SQL database is acceptable because ADR-001 kept one deployable. Ownership is still split by schema: catalog, ordering, billing. Application code in Catalog does not query ordering tables. A report that needs both is an explicit read model or a reporting schema fed by a copy, not an ad-hoc join hidden in a controller.

The transaction boundary is the module's own change. Ordering can insert the order row and the outbox row in one transaction. Ordering cannot, in the same transaction, update a table Billing owns and a table in an external warehouse. If you need that, you have a saga, which is day 10, or you have the wrong boundary and the two facts are actually one module.

The transactional outbox solves a specific failure: the order commits, the process dies, the message never leaves. You insert the message payload into an outbox table in the same commit as the order. A relay reads unpublished rows and sends them to Service Bus, then marks them published. The relay can crash after the send and before the mark, so the message can appear twice. That is allowed. Consumers dedupe. Do not 'solve' this by publishing to the bus before the commit. A crash then announces an order that does not exist.

Price is copied onto the order line at submit. Catalog can change the list price a minute later. The order keeps the price the buyer accepted. That copy is deliberate duplication. The alternative, reading Catalog live for every historical invoice, rewrites history.

Multi-tenant means every buyer company is a tenant. The tenant id comes from the authenticated session, never from a hidden field the client can edit. Every order query includes the tenant. A cross-tenant test tries to read company B's order with company A's session and must fail. SQL row-level security can be a backstop so a missed filter still returns nothing. It is not a substitute for the filter you can see in code review. A unique idempotency key is unique per tenant, not globally, unless you designed global keys on purpose.

CQRS means the write model and the read model are different shapes. Use it when the screen fights the transaction. Order history search across a year, with free text, does not belong on the orders table as a pile of

LIKE queries. A search index fed by the outbox is a read model. Getting one order by id is not CQRS. It is a query. Do not split read and write databases on day one to sound advanced.

```
One transaction in Ordering:
  insert into ordering.Orders (...)
  insert into ordering.Outbox (Id, Type, Payload, PublishedAt null)
commit
Relay, separate loop:
  read a batch where PublishedAt is null
  send to Service Bus
  set PublishedAt
Consumer:
  if this event id was already applied, return
  else apply and record the event id
```

Say this

Ordering is the only writer of an order. The price on the order is a copy taken at submit. The outbox row commits with the order, and the publisher is allowed to send twice. Every query is scoped by the tenant on the session. A search screen can have its own copy. It does not become a second writer of the order.

AI session

```
Here is my schema ownership sketch and outbox description.
Find any data with two writers.
Find a query that reaches into another module's tables.
Tell me what the buyer sees if the relay is down for an hour.
Do not invent a second database.
```

Evening artifact

1. docs/c4/data.md: schemas, writer, what is copied.
2. The outbox steps in your own words, including the duplicate.
3. The cross-tenant test you expect the team to keep.

Q1. How do you keep the write and the message consistent?

Answer. I write the order and an outbox row in the same SQL transaction. A relay publishes the row and can publish it twice if it crashes at the wrong time. Consumers are idempotent. I do not publish to Service Bus before the commit, because the message would outrun the data.

Say this as an architect

Search and mail read copies. They are not allowed to update the order.

Day 10 — Consistency without fairy tales

Outcome: you can draw the order states, say what happens when a message is delivered twice or out of order, and choose orchestration or choreography for one Harbor saga.

Time box (4 hours)

- 70 min: duplicates, ordering, sagas, and CAP below.
- 40 min: Kleppmann chapter 8, the failure modes, skim. Or Hard Parts on sagas.
- 70 min: state machine and one compensation.
- 30 min speak, 30 min AI, 10 min redraw the states.

Study material

Distributed systems fail in boring ways: the message arrives twice, the acknowledgement is lost, the receiver is slow, clocks disagree, a process dies between two writes. Harbor does not need a theorem to respect that. It needs a state machine and handlers that can run twice.

At-least-once delivery is the contract you should design for on Service Bus with peek-lock. The lock expires, another worker gets the message, both might have done the work if the first one finished and then failed to complete the message. Exactly-once is a phrase vendors use for a narrow mechanism. The effect you want, one mail and one invoice, is something you build with an idempotency key or a processed-event table. Say that plainly. Duplicate detection on the broker reduces accidents inside a window. It is not the design.

Ordering is not free. If two events for the same order must be applied in order, use Service Bus sessions keyed by order id, and accept less parallelism for that order. If the events commute, or if each event carries the full new status, you do not need sessions. A 'set status to Accepted' event that arrives after 'set status to Cancelled' is a bug unless the handler ignores a stale status. Prefer a state machine that refuses illegal transitions over a hope that the bus stays ordered.

Harbor order states you can defend: Draft is optional and local to the UI. Submitted means we have a durable row. Accepted means it passed credit and any reservation you decided was in the transaction. Rejected means credit or validation failed and the row exists so support can see it. Cancelled means an accepted order was undone by a deliberate compensation. Do not delete the row. Deleting destroys the audit and breaks idempotency.

A saga is a sequence of local transactions with a compensation or a retry when a later step fails. Two styles. Orchestration: one component, here Ordering, tells the next step what to do and stores the saga state. Choreography: each module reacts to events and emits the next. Orchestration is easier to see for two or three steps and one team. Choreography spreads the process across teams and makes the overall state harder to find. For Harbor, if inventory reservation is a second step outside the order transaction, orchestrate it from Ordering. If reservation fails, compensate: mark the order Rejected or Cancelled, release anything you held, and do not send the buyer a success mail. If Billing only needs to hear that an order was accepted,

that is choreography of a notification, not a saga. A reaction with no compensator is not a saga. It is a subscriber.

CAP, said carefully: when a network partition happens, a distributed data system cannot give you both perfect availability and perfect linear consistency. It is not a daily slogan and it is not 'you can only pick two of consistency, availability, and partition tolerance' as a product feature matrix. Partitions happen. Harbor's order write chooses consistency: if the database cannot be reached, the submit fails. The catalog read might choose a stale replica and be more available. Say which path chose which. Do not say 'we are AP' for the whole company.

```
Submit (orchestrated):
  commit order as Submitted, with outbox OrderSubmitted
  ask inventory (sync or a waited reply with a timeout)
  if reserved: set Accepted, outbox OrderAccepted
  if not: set Rejected, release nothing that was not held, outbox OrderRejected
Mail worker:
  on OrderAccepted, send once per (order id, Accepted)
  ignore duplicates
  do not mail on OrderRejected
```

Say this

The order has states, and illegal jumps are refused. The bus can deliver twice, so the mail worker records the event id. I orchestrate reservation from Ordering because there are two steps and one team, and I can show the saga row. I do not call that exactly-once. If SQL is partitioned away from the API, the submit fails rather than accepting an order we cannot store.

AI session

```
Here is my order state machine and the reservation saga.
Find a transition that loses an audit.
Find a step that charges or reserves twice if the message is replayed.
Say whether this is orchestration or choreography, and whether I hid the state.
Do not add a new broker.
```

Evening artifact

1. docs/c4/order-states.md with states, who may move them, and one compensation.
2. A sentence on what duplicate delivery does to mail.
3. A sentence on CAP that names only the submit path.

Day 11 — Identity and the ways an order submit is attacked

Outcome: you can threat-model submit order with STRIDE, and you can say which Entra setup Harbor uses for buyers and for staff.

Time box (4 hours)

- 80 min: STRIDE, OWASP API notes, and identity zones below.
- 30 min: OWASP API Security Top 10, the object-level authorization entry. And the Entra authorization-code page if day 6's skim was thin.
- 70 min: a threat model page for POST /orders.
- 30 min speak, 30 min AI, 10 min redraw the trust zones.

Study material

STRIDE is a checklist, not a ceremony. Spoofing: someone pretends to be a buyer. Tampering: they change the payload, especially price, quantity of a sku they should not see, or tenant id. Repudiation: a buyer denies they placed the order, and you have no durable record of who was on the session. Information disclosure: company A reads company B's order, or a log line prints a personal email into a shared sink without retention rules. Denial of service: a script submits huge carts or hammers the route. Elevation of privilege: a buyer token calls an ops endpoint, or a viewer role updates an order.

The OWASP API list changes editions. The item you must be able to discuss by its meaning is broken object level authorization. The call is GET /orders/{id}. You load the order by id and return it. The attacker uses their own valid token and someone else's id. The fix is the tenant and the user scope on every read and write, tested with a cross-tenant case. Hiding the id in a GUI is not the fix. A 404 for both 'missing' and 'not yours' avoids confirming that the other id exists.

Identity zones. Staff authenticate against the workforce Entra tenant. Buyers on a new system authenticate against Entra External ID. Existing products may still be on Azure AD B2C. Do not start a new B2C tenant in 2026 unless a control you require is missing from External ID; check Microsoft's current comparison the week you decide, and record it in the ADR. Two app registrations. The ops API rejects buyer tokens by audience, not by a role string the client sent.

Secrets. Managed identity for Azure SQL, Service Bus, and Key Vault. The Angular environment file is public. It may contain a public client id. It must not contain a client secret, a connection string, or an API key that grants data. User secrets and Key Vault references are for the server. Rotating a secret that was shipped in JavaScript means you already lost; assume it was copied.

A content security policy and a dependency audit belong in the same conversation as the token story. XSS is how a cookie session is ridden and how an in-memory token is stolen. The architect names the control: strict CSP, no secret in the bundle, HttpOnly cookie, anti-forgery on writes. npm audit and NuGet audit are signals. A failing audit on a transitive test package is not the same as a failing audit on the authentication library. The team needs a written rule for which severity blocks a release.

Threat	Harbor control
Spoofing	Entra, BFF session, cookie flags. No long-lived token in JavaScript.
Tampering	Server-side price. Tenant from the session. Anti-forgery on cookie writes.
Repudiation	Order row stores user id, tenant, timestamp. Audit is separate from logs. Day 24.
Information disclosure	Tenant filter and a cross-tenant test. 404 when the order is not in the tenant.
Denial of service	A rate limit on submit, a max page size, and a timeout on payment.
Elevation	Buyer audience cannot call ops routes. Authorization checked on the server, per action.

Say this

I threat-model the submit with a checklist, and the control I care about most is object-level authorization: a real buyer token must not read another tenant's order. Staff and buyers are different identity tenants and different audiences. Secrets stay off the Angular bundle. Managed identity reaches SQL.

AI session

```
Here is my STRIDE table for POST /orders and GET /orders/{id}.
Which control is actually a UI convenience?
Which threat has no test a developer could automate?
Where did I trust a value from the browser?
Do not invent compliance certifications.
```

Evening artifact

1. docs/security/threat-model-order-submit.md using the table, rewritten.
2. Identity note: workforce tenant, external tenant, what a buyer token cannot call.
3. One pipeline rule: the severity that blocks a release, and who may waive it.

Q1. How do you secure a multi-tenant API?

Answer. The tenant comes from the token or the server session, not the body. Every query is filtered by that tenant, and a test proves company A cannot read company B. Authorization is per action on the server. I return 404 when the id is outside the tenant. Row-level security in SQL is a backstop. The Angular app hiding a button is not a control.

Say this as an architect

I keep buyer identity and staff identity in separate Entra configurations so a buyer token fails the ops audience check.

Day 12 — An Azure shape you can operate

Outcome: you can choose a host for each Harbor container and map the choice to the Well-Architected pillars without quoting a price.

Time box (4 hours)

- 70 min: hosts and pillars below.
- 40 min: Azure Well-Architected Framework, the five pillar overviews. Skim the Architecture Center 'choose a compute' guidance if time remains.
- 70 min: host decision table for Harbor.
- 30 min speak, 30 min AI, 10 min redraw the hosts.

Study material

The five pillars are reliability, security, cost optimization, operational excellence, and performance efficiency. A design review that only names products misses the pillar it is spending. Harbor in one region spends reliability: you will have downtime when the region or the database has downtime, and you accepted that in exchange for cost and operational simplicity. Write that down. Do not discover it in an incident.

App Service is the default host for the BFF and the modular monolith. You get slots for a swap, a simple scaling model, and a place for managed identity. Choose it when the workload is request and response and the team does not want to run a cluster. Azure Functions, isolated worker, is the default for the notification worker: an event starts it, it can scale separately, and a poison message should not exhaust the API's threads. Container Apps is the step when you need a container, a scale rule, or several processes, and you still do not want to operate Kubernetes. AKS is justified when a platform team already runs a cluster and the workload needs those controls. Twelve people and a wholesale portal do not justify AKS as a learning project.

API Management sits in front of APIs that external partners call, when you need keys or OAuth for those partners, quotas, and a product catalog of APIs. The first-party Angular app does not need APIM on day one. The BFF is its facade. Adding APIM 'for the future' is a component you must patch, secure, and explain at 2 a.m. Revisit when a partner must call you without the BFF cookie. That is the day 29 case.

Azure Front Door and a WAF matter when the app is on the public internet and you want a global entry, TLS at the edge, and a filter for obvious abuse. They do not fix broken object-level authorization. One region still means the data is in one region. Front Door does not turn Harbor into a multi-region system.

Identity to Azure resources is managed identity. The App Service and the Function have identities. Those identities have roles on SQL, Service Bus, and Key Vault. No SQL password in a setting. Slots: production and staging. You deploy to staging, you smoke-test, you swap. Swap is not a data migration strategy. A bad migration still needs its own rollback, which is day 18.

.NET Aspire orchestrates the local run: the API, the worker, dependencies, and OpenTelemetry defaults. It does not choose your production host and it is not a cloud. Say that in one sentence so you do not get

marked as someone who thinks Aspire replaces Azure.

Multi-region is a new ADR, not a checkbox. Active-passive with a recovery point and a recovery time you can say out loud is the honest upgrade. Active-active needs a write strategy for orders so two regions do not accept conflicting submits. You do not have that strategy. Do not draw two regions 'for resilience' and leave the database as a single node with a dotted line.

Container	Host	Pillar you are being honest about
BFF and monolith	App Service, slots, managed identity	Operations stay simple. Reliability is one region.
Mail worker	Functions isolated	A poison message is isolated from the API process.
Data	Azure SQL, one region	Consistency of the order. Availability capped by that region and that service.
Messages	Service Bus	Durability of mail and integration, not of the order itself. The order is already in SQL.
Partner API, later	APIM only when a partner exists	Do not pay the operational cost on day one.
Cluster	Not AKS	Cost and operations. Revisit with a platform team and a need.

Say this

Harbor runs the API on App Service and the mail worker on Functions. One region, one Azure SQL, managed identity, slots for the app deploy. I am explicit that this is not 99.99 percent and not multi-region. API Management waits for a real partner. AKS waits for a platform team. Aspire is how we run it locally, not where it runs in production.

Interview trap

A diagram of twenty Azure icons for a team of ten. Every icon is a pager. If you cannot say who operates it, take it off the diagram.

AI session

```
Here is my Harbor host table.
Which service has no operator?
Where did I imply multi-region without a data strategy?
Which pillar did I ignore?
Do not add prices or SKU names you cannot tie to a requirement.
```

Evening artifact

1. docs/c4/azure.md, the host table and the one-region statement.
2. A revisit trigger for APIM and a separate one for a second region.
3. One sentence on what Aspire is for.

Day 13 — The API is a contract, not a controller

Outcome: you can specify submit order's status codes, idempotency, and versioning rule so two teams would build the same behavior.

Time box (4 hours)

- 70 min: errors, idempotency, pagination, and versioning below.
- 30 min: ASP.NET Core Problem Details documentation, skim.
- 80 min: write the contract for POST /orders and GET /orders.
- 30 min speak, 30 min AI, 10 min restate the status codes.

Study material

First-party Angular and a public partner do not get the same versioning story. The Angular app ships with the API. You can evolve with expand and contract: add a field, deploy the reader, deploy the writer, remove the old field later. You do not need /v1 in the URL until a client you do not deploy must keep working. When that partner exists, a URL version is the blunt tool they can actually use. Header versioning is easier to get wrong in caches and gateways. Pick URL versioning for the partner API and write it down.

Idempotency on create is an architecture decision because the buyer will double-click and the BFF will retry a lost response. The client sends an Idempotency-Key. Ordering stores it, unique per tenant, with the request hash and the order id. The same key and the same body return the same order and the same status. The same key and a different body return 409. A retry is not a second order. This is the same idea as the outbox consumer, applied to HTTP.

Status codes are a vocabulary. 201 when the order is stored, body includes the id. 400 when the input is invalid. 401 when there is no session. 403 when the session is real but the action is not allowed. 404 when the order is missing or not in this tenant. 409 for an idempotency conflict or an illegal state change. 429 when the rate limit trips. 503 when a dependency you chose to surface is down, with a retry hint only if the client can retry safely. Do not return 200 with an error field for these cases. The Angular client will treat 200 as success.

Error bodies use Problem Details, the shape ASP.NET Core already produces. A type, a title, a status, and a trace id the support person can paste. Do not return a stack trace. Do not return a SQL error. Validation errors can list fields. A payment decline is a domain result the UI can explain, and you decide whether it is 422 or a 201 with a Rejected order. Pick one. The rejected-order choice keeps an audit row. The 422 choice is simpler and easier to lose. Harbor keeps the row. Say so.

Pagination: offset pages lie when new orders insert above the cursor the user is walking, and large offsets get slow. A continuation token or a keyset on (created time, id) is the adult choice for an order list. Cap the page size. Sorting by an unindexed column is a performance incident waiting for a client.

OpenAPI is generated from the API and compared in CI. A breaking change is a failed check, or an explicit ADR if you truly must break a partner. The Angular data-access library is generated or handwritten against that contract, not against a controller you happened to read.

```
POST /api/orders
Idempotency-Key: required
Body: lines[{sku, quantity}]   # price and tenant ignored if present
201: order id, status Submitted or Accepted
409: key reused with a different body
400: validation, Problem Details
401 / 403 / 429: as above

GET /api/orders?pageSize&continuation
GET /api/orders/{id}  # 404 if other tenant
```

Say this

The first-party app versions by expand and contract. A partner gets a URL version when they exist. Create order requires an idempotency key, and the same key returns the same order. We keep a rejected order as a row so support can see it. List endpoints use a continuation token, not a page number we cannot honor.

AI session

```
Here is my POST /orders contract.
Find a case where the client retries and creates two orders.
Find a status code that will make Angular show success when the order failed.
Find a field I accept from the client and should ignore.
Do not redesign the BFF.
```

Evening artifact

1. docs/c4/api-orders.md with codes, idempotency, and pagination.
2. The versioning rule in one paragraph.
3. The Problem Details fields you will actually return.

Day 14 — Messaging decision and a red team

Outcome: ADR-002 is accepted in your portfolio, the threat model matches it, and you can survive a hostile reading.

Time box (4 hours)

- 30 min: re-read your integration table, outbox notes, and threat model. Fix contradictions before you write.
- 90 min: ADR-002, why Service Bus and an outbox, what you rejected.
- 40 min: speak the week: data owner, saga, identity, host, API contract.
- 40 min: AI red team, then edit the ADR.
- 20 min: redraw the path from button to mail with the laptop closed.

Study material

ADR-002 should be narrower than 'we use messaging'. The decision is: after the order commits, integration and mail go through an outbox to Azure Service Bus. Consumers are idempotent. Sessions are used only where order matters, named per flow. You rejected calling the email provider inside the submit request. You rejected a second database as a bus. You rejected claiming exactly-once. Consequences: mail can lag, and someone must watch the dead-letter queue. The revisit trigger: a consumer needs a million messages a second, an ordering guarantee Service Bus sessions do not give you, or a retention and replay story you measure and Service Bus cannot meet. Do not name Kafka as the revisit unless you can say which of those triggers fired.

The red team is the AI session plus you. A useful objection sounds like: the outbox relay is now a critical component and you have no SLO for lag. Or: the threat model says the tenant comes from the session, but the message payload also contains a tenant id that a bad relay could forge. The consumer must not trust the message blindly if the message can be published by more than the outbox. For Harbor, only the monolith's managed identity can send to that topic. The worker checks the order exists before it mails. A message alone is not proof.

```
ADR-002: Integrate after commit, through an outbox and Service Bus
Decision:
Mail and partner notifications are published from ordering.Outbox after the order transaction
commits.
Consumers dedupe on event id.
Submit does not call the email provider.

Rejected:
Send mail inside the HTTP request.
Publish to the bus before commit.
Exactly-once as a broker setting.

Revisit when:
```

```
Dead-letter or lag breaks an SLO you have written, or a measured volume or replay need Service Bus
cannot meet.
```

Say this

ADR-002 says the order is true when SQL commits, and the bus is how other parts hear about it. Duplicates are normal. The dead-letter queue has an owner. I will not put Kafka on the diagram until a revisit trigger is real.

AI session

```
Red-team ADR-002 against my principles, threat model, and submit sequence.
Find one place I can lose an order.
Find one place I can send two emails that the business would call a defect, and whether my dedupe
actually stops it.
Find one alert I named and one I forgot.
Do not rewrite the ADR. List the edits I should make.
```

Evening artifact

1. docs/adr/002-outbox-and-service-bus.md.
2. Threat model updated if the red team found a real hole.
3. Week score. Note the weakest of: data ownership, consistency, security, Azure shape, API contract.

Day 15 — Promises you can burn

Outcome: you can write an SLO for Harbor's accept path, and you can say where retries help and where they make an incident worse.

Time box (4 hours)

- 70 min: SLI, SLO, error budget, and stability patterns below.
- 40 min: the SLO chapter in Google's SRE book, or Release It! on timeouts and retries.
- 70 min: Harbor SLO sheet.
- 30 min speak, 30 min AI, 10 min restate the error budget in minutes.

Study material

An SLI is the measurement. An SLO is the target you promise for that measurement over a window. An error budget is the amount of failure the SLO allows. For availability, 99.9 percent over 30 days leaves about 43 minutes. That number is 30 days times 24 hours times 60 minutes times 0.001. If you cannot compute it on a whiteboard, do not quote the nines. 99.95 percent is about 22 minutes. 99.99 percent is about 4 minutes. Harbor's accept path, one region, one database, is a 99.9 conversation if the business needs a number at all. It is not a 99.99 conversation.

Separate the indicators. Availability of POST /orders: the fraction of requests that were not server failures, defined carefully so a 400 validation error is not downtime. Latency: p95 and maybe p99 of successful submits. Correctness: duplicate orders per thousand submits, which your idempotency key should drive toward none. Freshness of mail is a different SLO, looser, because you already decided mail is off the request. Do not average them into one green light.

An error budget policy is a human rule. When the budget is gone, you stop shipping changes that add risk, and you spend the time on the reliability work that burned it. Without a person who can say no, the policy is a paragraph. The architect writes the policy. The engineering manager enforces it. Name both.

Timeouts are how you stop waiting forever. Every outbound HTTP call has one. The timeout is lower than the user-facing budget if the call is on the request. Retries are how you survive a blip. Retry only when the operation is idempotent or you have a key that makes it so. Use a small number of tries and jitter so all callers do not stampede back at the same millisecond. Never retry a payment authorization because the response timed out. You do not know if the provider took the money. You reconcile. A circuit breaker opens when a dependency is failing, so you fail fast instead of filling the server with waiting requests. A bulkhead keeps a noisy dependency from using every thread. In Harbor the mail worker is already a bulkhead because it is another process. Inside the API, the payment client gets its own limit if payment stays on the request.

Polly, and the .NET HTTP resilience handler, are the tools. The interview does not need the pipeline syntax. It needs the policy: timeout, retry with jitter for idempotent GETs, no blind retry for payment, circuit breaker around a provider that is already returning errors.

The Angular side of the same decision: the submit button disables after the first click, the Idempotency-Key is generated once per attempt, and a failed read of the order list can show the last list with the time it was fetched. Do not clear the cart on a timeout until you know whether the 201 happened. Offer 'check latest orders' when the response is lost. That is the user-facing half of idempotency.

Say this

The accept path SLO I would start from is 99.9 percent availability and p95 under 400 milliseconds, both assumptions until measured, over a month. That is about 43 minutes of budget. Mail has a looser SLO. I retry idempotent reads. I do not retry a payment call that timed out. The UI keeps one idempotency key for the click so a timeout does not become two orders.

Interview trap

Promising 99.99 percent because it sounds serious, or putting a retry loop around every HttpClient including charges. Nines without a region and a data strategy are fiction. Retries without idempotency are duplication.

AI session

```
Here is my SLO sheet and retry policy.
Did I mix mail latency into the accept-path SLO?
Which retry can double-charge or double-order?
Is the error budget in minutes, and did I name a person who can stop risky releases?
Do not raise the nines.
```

Evening artifact

1. docs/quality/slo.md with two SLOs, the budget in minutes, and what you will not promise.
2. A five-line retry policy.
3. The Angular behavior when the submit response is lost.

Q1. What SLO would you set, and how do retries fit?

Answer. I set an SLI first: success and p95 latency of the accept path, not of mail. A starting SLO for this one-region system is 99.9 percent and a p95 target we label as an assumption until we measure. The budget is about 43 minutes a month, and someone has to be allowed to stop risky releases when it is gone. Retries are for idempotent calls, with jitter. A timed-out payment is reconciled, not clicked again by a loop.

Say this as an architect

The Angular client uses one idempotency key per submit so the user's double-click is safe too.

Day 16 — See the request from the browser to SQL

Outcome: you can say what you trace, what you count, what you log, and which three conditions page a human.

Time box (4 hours)

- 70 min: signals, cardinality, and correlation below.
- 30 min: ASP.NET Core OpenTelemetry overview, skim.
- 80 min: observability map for submit order.
- 30 min speak, 30 min AI, 10 min list the three pages from memory.

Study material

Logs, metrics, and traces are three different questions. A log is a discrete event you can read, such as 'order 123 accepted' with a trace id, not a novel. A metric is a number you aggregate over time: request count, error count, latency histogram, CPU. A trace is the path of one request across BFF, monolith, and SQL, made of spans. You need all three. A trace without metrics does not tell you if the error is rare. A metric without a trace does not tell you why this order was slow.

Cardinality is how you drown the bill and the tool. A metric label may be the route template, the status code, the tenant tier if you have a handful of tiers. A metric label must not be the order id, the user id, or the raw URL. Those belong on a trace or a log, sampled if volume demands it. Log ingestion is a cost driver. Logging the body of every submit will include customer data and a large bill. Log the ids, the decision, and the trace id.

Correlation: ASP.NET Core can emit OpenTelemetry traces and metrics, exported OTLP to Azure Monitor or another backend you already operate. W3C traceparent is the header that ties spans together. From the browser, you have two honest options. A real user-monitoring tool starts a trace in the page and propagates traceparent to the BFF. Or the BFF starts the trace and returns a request id header the UI can show in an error dialog. The second is enough for Harbor. Do not claim the Angular app is fully traced if you have not built the first.

The RED signals for the API are rate, errors, and duration. The USE signals for the App Service plan and SQL are utilization, saturation, and errors. Google's four golden signals are latency, traffic, errors, and saturation. You do not need three dashboards that say the same thing. You need one page a tired person can read: submits per minute, error ratio, p95, SQL CPU or DTU saturation, Service Bus dead-letter depth, outbox lag.

Page a human when the SLO is burning fast, when the payment provider or SQL is failing the accept path, or when the dead-letter depth or outbox lag crosses a line you set. Do not page on a single 500. Do not page on mail delay inside its looser SLO. The alert should link to the dashboard and say what 'good' looks like. An alert with no owner is a log.

Say this

Submit is traced from the BFF through SQL with OpenTelemetry, and the UI shows a request id when we fail. Metrics use the route and the status, not the order id. I page on SLO burn, on the database or payment path being down, and on dead-letter or outbox lag. I do not page on one exception, and I do not log the request body.

AI session

```
Here is my observability map and my paging list.
Which signal has unbounded cardinality?
Which alert will fire every night and get ignored?
What can I not see if the outbox relay stops?
Do not add a second observability vendor.
```

Evening artifact

1. docs/quality/observability.md: logs, metrics, traces, the three pages.
2. A line on what you will not log.
3. How a support person uses the request id.

Day 17 — Cost is an architecture constraint

Outcome: you can name Harbor's cost drivers and the assumptions a bill would test, without inventing a price.

Time box (4 hours)

- 60 min: drivers and questions below.
- 30 min: Well-Architected cost pillar, the principles page only.
- 80 min: assumption and cost-driver page.
- 30 min speak, 30 min AI, 10 min recite the drivers.

Study material

Architects lose trust by quoting a monthly figure they computed from memory. They gain trust by naming what moves the bill and what assumption the figure depends on. For Harbor the drivers are: the App Service plan's instance count and SKU, because it runs all month; Azure SQL's tier, because it also runs all month; Service Bus, which is usually smaller until you do something chatty; egress, if you move data out of the region; and log ingestion, which surprises teams who turn on verbose logging in production. The Functions worker is consumption-shaped and should not dominate unless mail volume is huge.

The unit of scale is an order, and behind it a tenant. A question you ask: what does one extra tenant cost if they submit ten orders a day versus ten thousand? If you cannot sketch the shape, you do not understand which tier is flat and which is per request. App Service and a provisioned SQL tier are mostly flat until you scale out or up. A per-request AI feature or a per-GB log pipeline is not flat. Call the difference out before someone adds 'AI recommendations' to the catalog page.

Degrade on purpose. If SQL is the cost center and the reliability center, you do not save money by running it too small to meet the SLO. You save money by not running a second region you cannot use, not keeping APIM for a partner who does not exist, not retaining debug logs for 90 days, and not sizing for a peak you have not measured. After launch, you read the bill and the metrics together and you right-size. Reserved capacity and savings plans are a finance conversation once the shape has been stable. Mention them as a follow-up, not as the design.

The Azure pricing calculator and the actual invoice are the sources. In a review you say: 'I would price this from the calculator with these assumptions, then check the bill after a month.' Then you list the assumptions: instance count, SQL tier family without a fake dollar amount, log volume in GB if you have a guess and the word assumption next to it, region, and no multi-region.

A cost decision is still an ADR when it changes the shape. 'We will not add a second region this year' is a cost and reliability decision. 'We will sample traces above this rate' is a cost and operability decision. Put the big ones in writing.

Say this

The bill will be mostly App Service and Azure SQL, plus whatever we log. I will not quote a price in the room. I will list the assumptions, open the calculator with the person who pays, and refuse a second region and a spare API gateway until a requirement forces them. Verbose logs are a cost incident as much as a security incident.

AI session

```
Here is my cost-driver page. I did not include prices on purpose.
Which driver did I miss for this design?
Which assumption, if wrong by ten times, changes the host or the tier?
Where did I optimize cost by weakening the SLO without saying so?
Do not fill in dollar amounts.
```

Evening artifact

1. docs/quality/cost.md: drivers, assumptions, what you will not run.
2. One degrade decision: what you turn off or delay when cost or load forces a choice.
3. Who signs the bill, from your day 1 stakeholder map.

Day 18 — Change the system without betting the company

Outcome: you can describe a strangler slice from a legacy .NET and Angular app, including the shared database and the rollback.

Time box (4 hours)

- 70 min: fitness functions, expand and contract, and the slice below.
- 40 min: Building Evolutionary Architectures, the fitness-function idea. Fowler's strangler fig article if you want the original metaphor.
- 70 min: one migration slice with a rollback.
- 30 min speak, 30 min AI, 10 min restate the rollback.

Study material

An evolutionary architecture is one you can change with a guided set of checks. A fitness function is that check, automated where you can. For Harbor's future modular monolith, a fitness function is a test that Catalog does not reference Ordering's infrastructure project. NetArchTest is one way to write it. A contract test that the order message still has the fields mail needs is another. A pipeline check that new HTTP clients set a timeout is another. You do not need twenty. You need the two that would have caught the last painful incident.

The legacy you should be able to discuss looks like this. An ASP.NET application on .NET Framework, a UI that is old Angular or a mix of MVC pages and Angular, and one SQL database in dbo that a reporting analyst queries with saved scripts. The business cannot stop taking orders. A rewrite-in-a-room for nine months is how this fails. A strangler moves one slice of behavior to the new system, leaves the rest, and routes only that slice.

Pick a slice that earns its keep and has an edge you can intercept. Example: new orders from the buyer portal go to the new Ordering module, while invoices remain on the old app. The old UI gets a link, or the new BFF routes that path. You do not migrate 'the customer entity' in the abstract. You migrate a journey.

An anti-corruption layer translates the old row into the new model at the edge, in one direction, until the old writer is dead. Old column names do not enter the new domain. If the old and new systems both write the same order row, you do not have a slice. You have a race. During the overlap, only one writer is allowed. The other reads.

Expand and contract is how columns change. Add the new column. Deploy code that can read it. Deploy code that writes it. Backfill. Switch readers. Remove the old column in a later release. Dropping the column in the same release as the new code is how you remove rollback.

The reporting analyst is a stakeholder, not an obstacle you route around. If they query dbo directly, they are a second consumer of the schema. You either support a view with a contract, or you give them a copy, and you do not drop their columns until they have moved. Put that in the plan or the plan is fiction.

Rollback: a feature flag or a route sends submits back to the old writer, which still works because you did not drop its columns and you did not let the new system change a row the old system cannot read. A flag that lives for a year is a permanent branch. Give it an expiry in the ADR. Data written only by the new system during the trial must be readable by the old system or explicitly in scope to discard. Say which.

Say this

I would strangle one journey, new buyer submits, and leave invoices on the old app. One writer of an order during the overlap. Expand and contract on the table so we can route back. The reporting scripts are a consumer I schedule, not a surprise. The flag expires. I do not fund a nine-month freeze to rewrite the whole estate.

AI session

```
Here is my strangler slice.
Who writes the order during the overlap?
What breaks for the reporting analyst?
What do I do with rows the new system wrote if we flip the route back on Friday?
Point at any step that drops a column too early.
```

Evening artifact

1. docs/reviews/strangler-orders.md, the slice, the single writer, the rollback, the analyst.
2. Two fitness functions you would put in CI after the new module exists.
3. An expiry for the route flag.

Q1. How would you modernize a legacy .NET and Angular system?

Answer. I would not rewrite it in one cut. I would move one journey, such as buyer submit, to the new ASP.NET Core module, keep a single writer, and leave the rest on the old app until that slice is proven. Schema changes expand and contract so the route can flip back. Anyone who queries the old database, especially reporting, is in the plan. A feature flag has an expiry.

Say this as an architect

I would add a fitness test so the new module cannot grow a dependency back onto the old web project.

Day 19 — Angular at architect altitude

Outcome: ADR-003 chooses how the buyer and ops UI are structured, and you can say when microfrontends are the wrong spend.

Time box (4 hours)

- 80 min: boundaries, state, performance, and accessibility below.
- 30 min: angular.dev pages on standalone components and signals, only to confirm the vocabulary you will use.
- 70 min: ADR-003 in your words.
- 30 min speak, 30 min AI, 10 min redraw the library boundaries.

Study material

At this altitude you do not debate a single component's change detection first. You decide what is allowed to depend on what, who releases the UI, and which budgets are real. The buyer portal and the ops portal are different audiences and different threat zones. Default in this plan: two Angular applications, so a buyer bundle does not contain ops screens, and a buyer host does not serve the ops app. The cost is two pipelines and a shared library to keep the visual language. The alternative, one app with lazy routes and a route guard, is acceptable when the team is small and the ops screens are not sensitive enough to force a separate host. That alternative is a decision, not a default you forget to mention. Pick one in the ADR and name the security consequence.

Inside an app, boundaries follow the domain. Feature areas or libraries for catalog, ordering, and billing. A shared UI library has no business rules. A data-access library owns HTTP to the BFF and the types in the contract. Feature code does not hand-build URLs in random components. Enforce it. In a single CLI workspace, a written rule and review may be enough. Nx, or another workspace tool, earns its place when several teams and many libraries make the graph too hard to see. Do not adopt Nx as identity. Adopt it when the boundary is already breaking.

State: signals are the default for state the component or the feature owns. RxJS stays at the edge where you truly have a stream: HTTP composition, typeahead, a socket. Do not convert every observable in a working app because the style guide moved. Do not introduce NgRx, or any global store, until you can name a bug that local state cannot fix, usually distant screens sharing a long workflow and going stale. A store is a cost in boilerplate and in concepts the next developer must learn.

Microfrontends, including module federation or native federation, let two teams release UI independently. They also split the design system, the auth story, the performance budget, and the local developer experience. Choose them when two release trains cannot share a repository and that pain is already here. Harbor's two squads can share a repo. The ADR says no. The revisit trigger is a team that must ship a portal area on a cadence the others block, and a platform person who will own the shell.

Performance is a budget, not a Lighthouse screenshot on your laptop. Set a target you will measure for the buyer home: transferred JavaScript on first load, and largest contentful paint on a throttled profile. An example target you may write as a target, not as a law: a shell that stays under a couple of hundred kilobytes compressed, then measure and adjust. Lazy-load ordering and billing. Do not lazy-load so aggressively that the submit path pays the delay. Server-side rendering helps a public catalog that must paint fast or be indexed. The authenticated order flow can stay client-rendered. Do not SSR the whole authenticated app as a slogan.

Accessibility is a quality attribute. WCAG 2.2 AA is the usual bar. The fitness function is an automated scan on the order path in CI, plus one keyboard pass a human still does on the submit flow. A button that only works with a mouse is a failed accept path for a class of users, not a polish item for later.

Zone.js and zoneless change detection: many production apps still run with Zone.js. Zoneless is a migration you schedule when change detection cost shows up in a profile, or when you are starting a new app and the current Angular documentation marks the path as supported for your major. Check angular.dev the week you write the ADR. Do not make zoneless the headline of your architecture review.

Say this

I split buyer and ops when the threat zone differs, and I keep domain libraries aligned with Catalog, Ordering, and Billing. Signals for local state, RxJS at the HTTP edge, no global store until we can name the stale-state bug. No microfrontends while two squads can share a repo. A performance budget and an accessibility check are fitness functions, not slides.

AI session

```
Review ADR-003, my Angular composition decision.
Does the security boundary match the deployable boundary?
Where did I add Nx, NgRx, SSR, or microfrontends without a trigger?
Which budget can a pipeline enforce?
Do not upgrade the Angular version in the answer.
```

Evening artifact

1. docs/adr/003-frontend-composition.md.
2. A sketch of apps and libraries, one page.
3. The performance target and the accessibility check, each labeled as a target until measured.

Q1. Would you use microfrontends?

Answer. Not for Harbor. The two squads can share a repository, and microfrontends would split auth, the design system, and the performance budget. I would split buyer and ops into two apps if we need a hard security and deployment boundary, and keep domain libraries inside each. I would revisit federation only when a team must release on a cadence the shared repo is blocking, and someone owns the shell.

Say this as an architect

I would rather enforce a library boundary with a review rule than add a runtime composition framework.

Day 20 — Governance people will still talk to you about

Outcome: you can say which decisions must come to a review, what the forum is for, and what is on your technology radar.

Time box (4 hours)

- 60 min: forum, triggers, and radar below.
- 30 min: Accelerate, the four metrics only: deployment frequency, lead time, change fail rate, time to restore.
- 80 min: write the triggers and a 15-item radar.
- 30 min speak, 30 min AI, 10 min recite the triggers.

Study material

Governance fails in two directions. A committee that must approve every library, which teaches people to route around you. An absence of decisions, which teaches every squad to pick a different serializer, a different auth library, and a different logging shape. The architect's forum is a short meeting on a fixed rhythm, one hour every two weeks, with a written page in advance. It is not a gate on the sprint.

Bring a topic when it matches a trigger: a new integration or a new data store; a new API that another team or a partner will call; an exception to a written principle; a step-change in cost; a new identity boundary; a production incident that was an architecture miss. Do not bring formatting, a single component, or a NuGet update that follows the existing standard. Those live in code review with the tech lead.

Definition of ready for the forum, one page: the context, at least two options, a recommendation, the quality scenario you are serving, the risk, and the revisit trigger. If the page is missing, the forum does not invent the design live. You send it back. That feels strict and it saves the hour.

A technology radar, in the Thoughtworks sense, has four rings. Adopt: the default, use it. Trial: use it in one place and learn. Assess: study it, do not put it in production yet. Hold: do not start new work with it. Harbor's radar should have about fifteen blips, not a catalog of the industry. Examples you should edit into your own voice. Adopt: modular monolith, BFF with cookie session, outbox, OpenAPI check in CI, managed identity. Trial: .NET Aspire for local orchestration. Assess: microfrontends, multi-region active-passive. Hold: new Azure AD B2C tenants, token in localStorage, AKS without a platform team, shared database as an integration style for new work.

The four Accelerate measures tell you whether the governance is hurting delivery. If lead time explodes after you introduce the forum, the forum is too heavy. If change fail rate is high because nobody reviews boundaries, the forum is too light or the fitness functions are missing. You do not need a metrics program this month. You need to know the four names and what a bad movement would mean.

Standards you write: principles, ADRs, API error shape, auth, logging. Standards you do not pretend are architecture: brace style, which an analyzer owns. Spending political capital on braces is how architects

become irrelevant while the data model rots.

Say this

The forum is one hour every two weeks for new integrations, new stores, identity, cost steps, and principle exceptions. It is not a gate on every story. The radar has a few items we will actually enforce. Adopt the outbox and the BFF. Hold localStorage tokens and a cluster we cannot operate. If the forum slows delivery without reducing failed changes, I shrink the forum.

AI session

```
Here are my review triggers and my 15-item radar.
Which trigger will fire on ordinary sprint work and annoy the team?
Which radar item contradicts ADR-001, ADR-002, or ADR-003?
Which Hold item has no reason a skeptic can read?
Do not add tools to look current.
```

Evening artifact

1. docs/principles/review-forum.md: cadence, triggers, definition of ready.
2. docs/principles/radar.md: fifteen items or fewer, four rings.
3. One sentence on which Accelerate metric would tell you the forum is too heavy.

Day 21 — Say the architecture in ten minutes

Outcome: a spoken briefing that covers the decision, the tradeoff, the risk, and the revisit trigger. No slides required.

Time box (4 hours)

- 40 min: read the script below, then close it.
- 90 min: write your own script from your artifacts, not from the sample.
- 50 min: record two takes. Watch one. Cut every sentence that does not contain a decision or a risk.
- 30 min: AI listens only as a critic of the script text.
- 30 min: third take, from memory, ten minutes or under.

Study material

A briefing is not a tour of Azure. Use this shape. One minute: who the buyer and the ops user are, and the scale assumption. Two minutes: the containers, including what you refused to add. Two minutes: ADR-001 and ADR-003, the monolith and the UI boundary. Two minutes: the accept path, the outbox, and the SLO with the number of minutes in the budget. Two minutes: the threat you designed for, tenant isolation and no token in the browser. One minute: the biggest risk and the revisit trigger. Stop. If they ask nothing, you are done. If they ask something, answer with a tradeoff, not a new product.

Sample, to be replaced by yours. 'Harbor lets about two thousand buyer companies submit wholesale orders, with a peak we are treating as about forty submits a second until we measure it. Staff and buyers are different identity tenants. The Angular apps talk only to a BFF. Tokens stay on the server. One ASP.NET Core application holds Catalog, Ordering, and Billing, with separate schemas, because two squads can still release together and the order must commit with its outbox. Mail is a Functions worker off Service Bus. We are in one region on App Service and Azure SQL, so I will not promise more than three nines, about forty-three minutes a month. The risk I watch is the shared process: a bad catalog deploy can hurt submit. I split a module out only if cadence, measured scale, or compliance says so. I would not put the token in localStorage, and I would not start on Kubernetes.'

> **Say this**
>
> That whole sample is the target length. If your version needs a diagram the audience cannot see, the words are still weak. Practice until the containers and the revisit trigger survive without the page.

AI session

```
Here is my ten-minute script.
Mark every sentence that is a tool name without a decision.
Mark every number I did not label as an assumption or a measurement.
Tell me the question a skeptical director will ask at the end.
Do not rewrite the script. I will.
```

Evening artifact

1. docs/reviews/briefing.md, your script.
2. Three cuts you made after the first recording.
3. Week score: SLO, observability, cost, migration, frontend, governance. Name the weakest.

Day 22 — Case: design Harbor from a blank page

Outcome: in 90 minutes on paper you can produce a design another architect can attack. You do this before you reread your own documents.

Time box (4 hours)

- 15 min: read only the brief below. Do not open your docs.
- 90 min: paper. Context, containers, data owners, accept path, one failure. No laptop.
- 40 min: compare with your week 1 and week 2 docs. Note what you forgot, not what you would add.
- 30 min: speak the design.
- 30 min: AI as a hostile reviewer of the paper, typed in after.
- 15 min: one-page writeup in docs/reviews/case-harbor.md.

The brief

Harbor Wholesale wants a portal. About 2,000 buyer companies, about 15,000 users, a peak you may assume at about 40 order submits a second. Eight to twelve engineers, two squads. One Azure region. Card data stays at a payment provider. Buyers and staff must not share one identity pile. Ops staff release credit holds. Confirmation mail can be a minute late. Reporting today is a person with SQL scripts, and they will exist on day one of the new system only as a stakeholder you must not surprise. The business wants 'microservices and Angular' in the first sentence of the proposal. You may tell them no.

What a strong paper includes

- Actors and the two identity zones.
- Containers you can operate, and a short reject list: no AKS, no token in the browser, no shared database with the warehouse.
- Ordering as the only writer, price copied at submit, outbox in the same transaction.
- The SLO as an assumption, with the 43 minute translation if you offered three nines.
- One failure: SQL down rejects the submit; mail down does not. Or payment timeout is not retried.
- The sentence you would say to the person who asked for microservices.

A weak paper lists Azure products, draws five services because the brief said microservices, and never says what happens when the bus is down. If that is what you drew, that is the finding. Write it at the top of the notes. Do not quietly correct the paper and pretend the first hour went well.

AI session

```
You are a principal engineer reviewing a hiring exercise.
Here is my paper design for Harbor, transcribed.
Ask me six follow-up questions, one at a time is not required, all six now.
At least two questions must attack data ownership, identity, or failure.
```

```
Then score decision, tradeoff, and operability from 0 to 2 each, with one sentence of evidence.
Do not redesign it for me.
```

Say this

I drew a modular monolith, a BFF, and a separate mail worker. I rejected a microservice per module because the team is twelve people and the order needs a local transaction. The design is operable in one region, and I said what the buyer sees when SQL is down.

Day 23 — Case: the legacy system and the person who funds it

Outcome: a one-page modernization plan a director can fund, including what you will not rewrite this half.

Time box (4 hours)

- 20 min: the brief only.
- 80 min: the plan on paper, then typed.
- 40 min: a spoken version in four minutes for a non-engineer.
- 40 min: compare with day 18 and fix the plan, not the story you wish you had.
- 30 min: AI critic. 10 min: recite the rollback.

The brief

The current Harbor is ASP.NET on .NET Framework, plus an Angular app several majors behind, on one SQL Server database. A reporting analyst runs weekly scripts against dbo. Orders, invoices, and catalog live in the same tables the UI writes. Revenue cannot stop. You have one team of eight and two quarters, not a blank check. A director has asked for a rewrite on .NET 10 and the current Angular, 'so we are future-proof', and wants a date when the old system is off.

Study material: the plan that gets funded

Lead with the risk they already feel, not with the framework version. Orders are hard to change because the UI, the reports, and another team's extracts share one set of tables. The rewrite pays for itself only if a slice of order-taking becomes safe to change. Future-proof is not a date.

Scope of the first half: new buyer submit on the new stack, one writer of those orders, old system still does invoices and the rest. Reporting gets a view or a copy with a named owner and a date, because their scripts are a production dependency. Schema change is expand and contract. The route back to the old submit stays until the new path has lived through a peak week. The flag expires at the end of the second quarter, at which point you either cut over or you write a new ADR. The old runtime does not get a fantasy shutdown date in month one. It gets a condition: no remaining writer, reports moved, rollback no longer required.

Cost language a director can use: one team, no second product, no Kubernetes, a thin slice, a rollback. The thing you need from them is cover when someone asks why invoices are still old, and an introduction to the person who owns the reporting scripts. The thing you will not do is freeze features for six months to rebuild the catalog.

Say this

I can put new buyer orders on .NET 10 and a current Angular app in two quarters without stopping revenue. Invoices stay on the old system on purpose. We keep a way back until a peak week goes well. I need the reporting scripts treated as a consumer with a date. I will not give you a shutdown date for the whole legacy estate until nothing important writes to it.

AI session

```
Read this as a director who wants a shutdown date and hates risk.
Where is the date I asked for, and did the architect replace it with a condition I can understand?
What still takes revenue down if it fails?
What am I being asked to decide, in one sentence?
Point at jargon I would not repeat to my board.
```

Evening artifact

1. docs/reviews/case-legacy.md, one page, no framework lecture.
2. The condition that replaces the shutdown date.
3. The ask of the director, two sentences.

Day 24 — Case: mail, audit, and search are not one database

Outcome: you can separate three needs that teams often smash into SQL Server or into 'the logs'.

Time box (4 hours)

- 30 min: the brief.
- 90 min: three stores, three writers, three failure modes.
- 40 min: speak it.
- 40 min: AI critic.
- 20 min: write docs/reviews/case-mail-audit-search.md.

The brief

After submit, Harbor must email the buyer, keep an audit that can answer 'who accepted this order' six years from now, and let ops search last year's orders by company name and sku. A teammate proposes: write it all to the application log, and add three more indexes to the orders table for search. Another proposes event sourcing the entire order history because they read a blog. You have Azure SQL, Service Bus, and the option of a search index if you can justify it.

Study material

Mail is a notification. It can fail, it can be retried, it can be late. It is the outbox and the worker you already designed. It is not the source of truth. If the mail says Accepted and the database says Rejected, the database wins, and you have a bug in the worker to fix.

Audit is a business record. It answers who did what to which order, when, from which tenant. It must survive a log-retention policy that deletes debug logs after 30 days. Store it on purpose: a table Ordering appends to in the same transaction as the state change, or a dedicated audit store with a retention rule the business signed. Six years is a business requirement. Write it down and ask legal or the director if you invented the number. Do not invent six years in the interview if they did not say it. The brief said six. In real life you ask. The audit row is not an application insight log. Different access control, different retention, different reason.

Search is a read model. The orders table serves the transaction: get by id, enforce uniqueness, keep the state machine honest. Searching a year of text across company name and sku with a pile of indexes will fight the writes. A copy, filled from the outbox, can sit in a search index or in a table shaped for the query. It can be seconds or minutes behind. Ops must see a 'as of' time, or they will argue with a buyer about a phantom status. The search store is not a writer of the order. If someone 'fixes' a row only in search, the next event will overwrite it, or worse, you now have two writers.

Event sourcing replaces the order row with a fold of every event. It is justified when you truly need to reconstruct past states as a product feature and you accept the operational cost. Harbor needs the current state, an audit trail, and a search copy. That is an order row plus an audit append plus a projection. Selling

full event sourcing here is a different system. Say no, and say what you would revisit: a requirement to answer arbitrary historical questions that an audit table cannot, measured, not imagined.

Need	Store	If it is down or stale
Truth of the order	ordering.Orders	Submit fails. We do not accept what we cannot store.
Who did it	Audit table, long retention, same transaction as the change	The state change should fail too, if you required them in one transaction. Do not rely on logs.
Find last year's orders	Read model fed by the outbox	Search is stale or unavailable. The buyer can still open a known order id. Show the lag.
Email	Worker and the provider	Order stands. Alert on the dead letter. Do not roll back the order because mail failed.

Say this

The order row is the truth, the audit row is the legal memory, the search index is a copy that may lag, and the log is for diagnosis. I will not search by adding every reporting index to the transactional table, and I will not event-source the order because we need an audit.

AI session

```
Here is my split of mail, audit, and search.
Where did I still use the log as a system of record?
Can the audit row commit if the order rolls back? It must not.
What does ops see if search is ten minutes behind?
Is event sourcing actually required? If I included it, attack it.
```

Day 25 — Write the decision for someone who will not read the ADR

Outcome: a one-page memo that recommends staying a modular monolith this half, in language a director can repeat.

Time box (4 hours)

- 30 min: read the sample structure, not to copy the sentences.
- 80 min: write the memo from your ADR-001.
- 40 min: read it aloud. Cut adjectives.
- 40 min: AI plays the director.
- 30 min: a second memo, half a page, on a decision from your real career, with names removed.

Study material

A director memo has five parts. The decision in two sentences. The problem that made it necessary. The options, including the one they hope you will pick. The consequence in cost, risk, and calendar, without a fake precision. The ask: what you need them to say yes to. Put the ask at the top or the bottom, not in the middle of a diagram.

Sample shape, compress and replace. 'Decision: Harbor stays one application with hard module boundaries this half. We will not split Catalog, Ordering, and Billing into separately deployed services. Problem: we have two squads and one database transaction that must stay intact, and we have no team whose job is a cluster. Options: split now, stay one deployable with enforced modules, or keep one project with no boundaries. Splitting now spends the half on pipelines and partial failures instead of on the buyer journey. No boundaries means the next rewrite is already scheduled. Consequence: a bad release can still affect the whole API, and we cannot scale one module by itself. We accept that until a measurement or a team split says otherwise. Ask: agree that microservices is not a success metric for this half, and that a module split needs a new decision when a trigger fires.'

Disagreement is part of the job. You do not win by a longer document. You restate their goal, you show the cost of their option, and you offer the trigger that would make you join them. If they overrule you, you record the decision, the risk you named, and who accepted it. You do not sabotage the split after the fact, and you do not pretend it was your idea. An overruled architect who keeps a private 'real architecture' is a source of shadow systems.

Say this

I recommend we stay one deployable this half. Splitting services is the option I rejected because it spends the team on failure modes before we have a separate owner for any module. If a squad must release alone, I will bring back a split decision. I need you to treat that as the plan, not as a delay.

AI session

```
You are the director who asked for microservices.
Read my one-page memo.
What am I deciding?
What do you need from me?
Which sentence sounds like I am protecting my preference instead of the deadline?
Reply in plain language. Do not rewrite the memo.
```

Evening artifact

1. docs/reviews/memo-no-microservices.md.
2. A half-page memo from your own eight years, sanitized.
3. The sentence you will use when you are overruled.

Day 26 — Run the review instead of winning it

Outcome: a 60-minute agenda and a list of questions that expose a missing quality attribute without humiliating the author.

Time box (4 hours)

- 40 min: the agenda and the question list below.
- 80 min: review a design. Use a teammate's real design if you may, or review your own day 22 paper as if a colleague wrote it. Write comments in the form of questions.
- 40 min: practice the opening and the closing out loud.
- 40 min: AI role-plays a defensive author. You only ask questions.
- 20 min: save the agenda and the questions.

Study material

The review exists so the next incident is cheaper. It does not exist so the architect is the smartest person in the room. The author presents. You facilitate. If you take the pen in minute five, you have stolen the design and the learning.

Sixty minutes. Five: context and the decision they want. Ten: options they rejected. Fifteen: the quality scenarios and the failure they are willing to show the user. Ten: data ownership, identity, and what the client is allowed to decide. Ten: cost drivers and who is on call. Ten: the decision sentence, the revisit trigger, and the open risks. If the definition-of-ready page is missing, you spend five minutes saying so and you stop. Reschedule. Do not design it for them in the remaining time.

Questions that work. What happens to the buyer if this dependency times out? Who is the only writer of this row? What did we reject, and who is unhappy? What number is an assumption? What will page someone? What is the rollback if we are wrong on Thursday? Which principle does this exception bend? What would make you revisit this in six months? Where is the tenant taken from? You can run a whole review on those sentences.

Questions that waste the hour. Why not Kafka? Have you considered a pattern I like? Can we rename this class? Why is this not using the library I wrote? Save those for a conversation that is not this meeting.

Close by reading back the decision, the risk, and the owner. Send the notes the same day. An undocumented review is a conversation the team will relitigate next sprint.

Say this

I will keep the review to an hour and send you back if the options and the failure mode are not on the page. I will ask who writes the data and what the user sees when the dependency dies. I will not redesign it in the meeting. I will read the decision back before we leave.

AI session

```
Role-play a senior developer who designed five microservices for Harbor and is proud of it.
I will only ask questions. After my questions below, answer in character, then tell me which of my
questions was an attack instead of a question.
My questions:
What fails for the buyer if service three is down?
Who writes the order row?
What did you reject?
What is the rollback on Thursday?
```

Evening artifact

1. docs/reviews/agenda.md.
2. Ten questions in your own words.
3. Notes from the practice review, including one question you would rephrase.

Day 27 — The portfolio and the stories only you can tell

Outcome: a README a hiring manager can read in ten minutes, and five stories from your career written as decisions.

Time box (4 hours)

- 40 min: the README structure below.
- 70 min: write the README so it links your real artifacts.
- 80 min: five stories. System, decision, alternative, consequence, what you would reopen.
- 30 min: AI checks that each story has a tradeoff. It must not invent metrics.
- 20 min: read one story aloud in two minutes.

Study material

The portfolio is the harbor-architecture repository plus a README. The README says who you are in six lines, then links: principles, context, containers, three ADRs, SLO, threat model, the legacy memo, the briefing script. A manager will open two of those. An architect interviewer will open the ADRs and try to break them. Make the links true. A link to an empty file is worse than no link.

Do not put your employer's private diagrams, customer names, or revenue in the repo. Say 'a wholesale portal' or 'a line-of-business app' and change the numbers if the real numbers are confidential. You can still tell the truth about the decision.

Stories from eight years beat Harbor, because Harbor is a case and they know it. Write five. A production incident and the minute you chose rollback or fix-forward. A design you pushed back on. A migration slice, even a small framework upgrade, and what you refused to rewrite. A data or query problem where you changed the shape, not just an index. A person you grew, and what they now decide without you. Each story is one short page. If you do not have a number, do not create one. 'It was slow enough that support saw it every morning' is honest. 'We improved latency by 63 percent' without a measurement is how panels lose trust.

The architect version of a story is not 'I worked hard'. It is 'we had two options, I recommended this, the cost was that, and the signal I watched afterward was this'. Practice stopping. Two minutes is enough. The follow-up is where depth shows.

Say this

The repository is how I make architecture decisions visible: context, ADRs, and the quality I would measure. The stories are from systems I actually shipped. I will tell you the option I rejected. I will not invent a percentage.

AI session

```
Here are five stories. For each, say whether you can find: a decision, a rejected option, a
consequence, and a way I would know it failed.
Flag any number that looks invented.
Flag any story that is really a list of technologies.
Do not improve the prose. Do not add achievements I did not write.
```

Evening artifact

1. README.md at the root of the architecture repo.
2. docs/reviews/stories.md with five stories.
3. One story you will tell if you are asked for only one.

Day 28 — Mock panel

Outcome: a scored 45-minute design and a 20-minute story conversation, with notes of what broke.

Time box (4 hours)

- 20 min: set up. Paper, a timer, the scoring sheet. Phone on record if you can stand to hear it.
- 45 min: design, laptop closed, AI silent.
- 20 min: two stories, timed.
- 30 min: transcribe the design enough for the critic.
- 40 min: run the panel prompt. Write the scores it gives and, more important, the scores you give yourself where you disagree.
- 25 min: a repair list of at most three items for day 29. More than three means you are avoiding the worst one.

The design prompt, unseen until the timer starts

A regional distributor wants a portal for their retailers to place replenishment orders. About 500 retailers, peak of about 10 submits a second, a team of six, Azure, .NET and Angular. They also fax orders today and want that to stop. Warehouse staff must see today's orders. Design the system. Call out what you will not build in the first release.

This is Harbor with the numbers changed on purpose. If you cannot redraw it under new numbers, you memorized a diagram. The fax line matters: it is a legacy input, an anti-corruption problem, and a temptation to keep a shared spreadsheet forever. Say what happens to fax in release one. A reasonable answer: release one is the portal for the largest retailers, fax remains as a manual entry screen for staff, and you do not build optical character recognition as architecture.

Panel prompt

```
You are a technical architect interviewing a candidate with eight years of .NET and Angular.
You have their transcribed design and two stories below.
Score 0, 1, or 2 on each: decision quality, tradeoff, operability, security and data ownership,
communication.
Ask eight follow-up questions they did not answer.
Name the single gap that would stop you hiring them as a technical architect this quarter.
Do not coach warmly. Do not redesign the system.
Be specific. Quote their words.
DESIGN:
(paste)
STORIES:
(paste)
```

Services-firm panels often add a workshop: assumptions, risks, and a rough phase plan. Product-firm panels often stay on the whiteboard and then ask how you would roll it back. If you know which room you are

walking into, add ten minutes in that style after the 45. Do not skip the failure mode in either room.

How you score yourself

- 2: you named the decision, the rejected option, and the user-visible failure.
- 1: you named the decision, and the tradeoff arrived only when pressed.
- 0: you listed technologies, or you agreed with every interruption.
- A total of 7 or more out of 10, with no zero on data ownership, is a pass for this mock. A beautiful diagram with a zero on security is not.

What the model must not decide

Whether you are 'ready for Google' or any other brand. Whether your stories are good enough to claim a title. Use the model to find holes. You decide what to repair. If the model flatters you, the prompt was too soft. Run it again with the line 'be specific' and a demand to quote you.

Day 29 — Repair, then a different case

Outcome: the worst gap from the mock is smaller, and you can design a partner API that must not reuse the browser session.

Time box (4 hours)

- 70 min: repair only the top gap. If it was data ownership, rewrite that page. If it was speaking, record the briefing again. Do not 'lightly review everything'.
- 20 min: the partner brief.
- 70 min: paper design.
- 30 min: speak.
- 30 min: AI scores only the partner case.
- 20 min: note what changed since day 22.

The brief

A large buyer wants to place Harbor orders from their own ERP, not from your Angular app. They can call an API. They cannot be given a staff password. They must not see another tenant. You expect tens of calls a second from them at peak, in bursts. They will get the contract wrong at least once in the first month. Decide what you add, what you reuse, and what the Angular BFF must not become.

Study material

Reuse Ordering, the idempotency store, the tenant rules, and the outbox. Do not reuse the cookie session. A partner uses an application identity: client credentials or a similar OAuth client flow against Entra, their own app registration, a credential they can rotate. Authorization says which tenant that client may act for. One client must not submit for every tenant because it knows an id. That mapping is server-side.

This is the moment API Management, or a carefully limited gateway, earns a box. You want a key or a token you can revoke, a quota so a bug in their ERP does not consume the accept path, and a versioned contract. The BFF stays the browser's door. If you route the partner through the BFF's cookie endpoints, you will invent a service account password in a hurry. Write the no.

Version the partner contract with a URL version from the start, because you cannot deploy their ERP. Expand and contract still applies inside a version. Give them Problem Details and a request id. Rate-limit with 429 and a documented retry they can honor. Their retries must send the same idempotency key or they will duplicate orders and blame you.

The Angular app does not get a special bypass around the new rules. If both doors write orders, they write through the same Ordering module. Two doors, one writer.

Say this

The partner gets their own client credentials, a tenant mapping we control, a versioned contract, and a quota. They use the same Ordering module and the same idempotency rules. They do not get a browser cookie, and the BFF does not grow a back door for them. API Management is justified now because a partner exists.

AI session

```
Score my partner-API design from 0 to 2 on decision, tradeoff, and security.
Did I reuse the cookie?
Is there still one writer of the order?
What happens when they retry without an idempotency key, and did I say it?
Quote the weakest sentence.
```

Day 30 — The first 90 days in the seat

Outcome: a personal operating plan for the job, not another diagram. You know what you will not redesign in the first month.

Time box (4 hours)

- 40 min: read the 90-day shape below and edit it to the company you are actually joining or targeting.
- 80 min: write your plan, including a listening list and the first fitness function.
- 40 min: speak the 30-day briefing one last time, from memory.
- 40 min: AI critic on the plan, looking for heroics.
- 40 min: close the portfolio. Fix broken links. Write the date.

Study material

The failure mode of a new technical architect is a target architecture in week two. The system you just joined has scars, political owners, and a release train. Your first job is to see it. Days 1 to 30 of the role, not of this course: read the last five incident reports, read the deploy pipeline, draw the real containers from the repositories rather than from the old slide, and have eight conversations. The tech leads, the person on call, the person who pays the bill, security, the product owner, support, the analyst with the scripts, and one engineer who is quietly holding a module together. Ask what they are afraid to change. Do not present Harbor.

Days 31 to 60: one decision the teams already asked for, written as an ADR, small enough to merge. One fitness function in CI that would have caught a real incident or a real boundary violation. One review forum that actually happened, with notes. If you cannot get a fitness function merged, you do not have the relationships yet. Go back to listening. Do not add a second initiative to hide that.

Days 61 to 90: the forum has happened twice. The radar has fifteen items or fewer and at least one Hold that annoyed someone, which means it is real. You have declined a rewrite or a platform in writing, with a revisit trigger. You can point to a decision someone else used without calling you. That is the job. A diagram nobody used is a hobby.

Keep a depth spike so you do not become a person who only reviews. For you that spike is ASP.NET Core, data access, and Angular. One day a week you read a pull request properly or you pair on a hard defect. The 10-day tech lead guide remains the refresher when a panel or a production bug drops below architecture altitude. Use it. Do not spend the first month rebuilding it.

AI in the job: the same rules as this course. The model may attack an ADR you wrote. It may draft a threat-model skeleton you then correct against the real system. It may not set the SLO, choose the region strategy, or write a performance number into a customer proposal. Paste constraints. Ask for the objection. Redraw from memory. Do not paste your employer's secrets into a personal account.

Say this

In the first month I will draw the system that exists and listen to the people who run it. In the second month I will land one decision they already need and one check in the pipeline. In the third month I will have declined something expensive in writing. I will not arrive with a target architecture named after a reference diagram.

AI session

```
Here is my 90-day plan for the architect role.
Find the heroic goal that a new person cannot finish.
Find a week where I am presenting instead of listening.
Find where I took implementation work away from a tech lead.
Tell me the smallest fitness function that would still count.
```

Evening artifact

1. docs/reviews/first-90-days.md.
2. The listening list, eight conversations, with the question you will ask each.
3. The first fitness function, described so a tech lead could add it without you.
4. A note of the mock scores from day 28 and what day 29 changed.

Q1. What would you do in your first 90 days?

Answer. I would not announce a new architecture. I would spend the first month on the real system and on the people who are afraid to change it. In the second month I would ship one ADR the teams already need and one automated fitness check. By the third month the review forum should have met twice, and I should have declined at least one expensive rewrite in writing, with a trigger that would reopen it.

Say this as an architect

I would still review code one day a week. An architect who cannot read the pull request will not see the boundary break.

Appendix A — Prompt library

Use these on your own artifacts. Replace the angle brackets. Keep the session prompt from the front of this document as the first message when the chat has forgotten the constraints.

Critique an ADR

```
Constraints: <team size, scale assumption, region, data store, identity>.
ADR below. Do not rewrite it.
1. Decision in one sentence, as you read it.
2. The strongest objection.
3. A consequence I listed as positive that is also a cost.
4. Whether the revisit trigger is observable.
5. Sentences I should delete because they are slogans.
ADR:
<paste>
```

Missing quality attribute

```
Here is a design. Name the quality attribute I did not cover.
Write the missing scenario as source, stimulus, environment, response, and measure.
If the measure cannot be observed, say so.
Do not add a product to fill the gap.
DESIGN:
<paste>
```

Threat model starter, then you delete the nonsense

```
Threat-model this flow with STRIDE.
For each threat: one sentence, one control that is a server-side check or a test, and one control
that is only a UI convenience.
Flag anything you are unsure about instead of inventing a CVE or a compliance claim.
FLOW:
<paste>
```

Mock interviewer, short

```
Interview me for a technical architect role on .NET and Angular.
Ask one question. Wait. I will answer in a later message.
After my answer, score 0 to 2 for decision, tradeoff, and consequence.
Then ask a follow-up that attacks the weakest of the three.
Do not lecture. Do not move to a new topic until I ask.
```

Director translation

```
Turn this ADR into a decision a director can read in two minutes.
Keep my decision. Do not soften a risk I named.
```

```
Cut product names that are not required to understand the cost.
End with the ask.
ADR:
<paste>
```

Boundary check in Cursor

```
Look at principles.md and the ADR I have open.
List contradictions only.
For each: quote both files.
Do not edit the files.
Do not propose a new architecture.
```

If you use Cursor for that last prompt, open the two files so the tool can see them. Then you still edit by hand. A clean contradiction list is the value. An automatic rewrite is how the principles drift to match the latest draft.

Stop prompts

- Do not ask which cloud is best.
- Do not ask for a full production code base for Harbor. A sketch that proves the boundary is enough. A generated solution you have not run is a liability in the portfolio.
- Do not ask the model to score you 'ready' or 'not ready' for a job. Ask it to quote the missing tradeoff.
- Do not paste an employer repository into a personal tool. Bring the shape, not the source, unless the company tool is approved.

Appendix B — Templates

ADR

```
# ADR-000: <decision in the title>
Status: Proposed | Accepted | Superseded by ADR-000
Date:

## Context
<constraints, not a history essay>

## Decision
<what we will do>

## Options rejected
<why each lost>

## Consequences
<good and bad>
```

```
## Revisit when
<observable trigger>
```

Quality scenario

```
Quality:
Source:
Stimulus:
Artifact:
Environment:
Response:
Response measure:
Assumption or measurement:
What we give up:
Revisit when:
```

One-page decision

```
Decision:
Problem:
Options:
Consequence and risk:
Ask:
```

Review notes

```
Date:
Author:
Decision requested:
Options rejected:
Failure the user sees:
Data owner:
Open risk:
Revisit trigger:
Who accepted:
```

Appendix C — Rapid answers

Speak the answer, then stop. Add the lead sentence only if they ask why.

Q1. What is your default for a new .NET and Angular product with one team?

Answer. A modular monolith for the domain, a BFF in front of the Angular app, and a separate worker only for work that must fail independently, such as mail. One database with schema ownership. Managed identity. One region until the SLO and the business say otherwise.

> **Say this as an architect**
> I write the revisit trigger at the same time, or the default becomes a religion.

Q2. When do you split a service out?

Answer. When a team must release on its own cadence, when a measured hotspot cannot be fixed inside the process, or when a compliance boundary forbids sharing the host. Not because the diagram has too few boxes.

> **Say this as an architect**
> Mail can be a worker earlier, because the failure domain is already different and the user is not waiting.

Q3. How do you handle a double submit?

Answer. An idempotency key, unique per tenant, stored with the order. The same key and body return the same order. The same key and a different body return 409. The UI generates the key once per attempt.

> **Say this as an architect**
> The outbox consumer uses the same idea for messages, because the broker is at least once.

Q4. Where does authorization live?

Answer. On the server, on every read and write, using the tenant and the user from the session or token. The Angular app can hide a button. That is not the control. A cross-tenant test is part of the definition of done.

> **Say this as an architect**
> I use 404 when the id is outside the tenant so we do not confirm it exists.

Q5. What do you put in the browser?

Answer. An HttpOnly Secure cookie issued by the BFF, plus an anti-forgery token on writes. Not the access token, and not a secret in the Angular environment file.

> **Say this as an architect**
> If we skip the BFF, the token stays in memory and I state the XSS risk instead of pretending localStorage is fine.

Q6. How do you choose between App Service, Functions, Container Apps, and AKS?

Answer. App Service for the request API, Functions for the event worker, Container Apps when we need containers without operating a cluster, AKS when a platform team already runs one and the workload needs it. Harbor does not start on AKS.

> **Say this as an architect**
> Every choice is a pager. I can say who answers it.

Q7. What is your availability target?

Answer. I do not pick nines from a hat. For a single-region Harbor I would discuss 99.9 percent on the accept path, about 43 minutes a month, as a starting SLO, and I would label it an assumption until we measure. I would not offer 99.99 percent without a second region and a data strategy.

Say this as an architect

Mail is a different, looser SLO. I do not hide it inside the accept number.

Q8. What is a fitness function?

Answer. An automated check that an architecture decision still holds. A test that one module cannot reference another's tables, a contract check on the order event, a timeout required on new HTTP clients, an accessibility scan on the submit route.

Say this as an architect

I would rather have two that match real incidents than a dashboard of twenty nobody owns.

Q9. Would you use NgRx?

Answer. Not by default. Signals for feature state, RxJS at the HTTP boundary. A global store when distant parts of a long workflow go stale and we can name that bug. The store is a cost.

Say this as an architect

The larger Angular decision is the boundary between buyer and ops, and between catalog and ordering, not the state library.

Q10. How do you work with AI as an architect?

Answer. I write the decision. The model attacks it: missing quality, weakest objection, fictional numbers. I do not paste secrets, I do not accept a price it invented, and I redraw the design from memory. If I cannot, I do not understand it yet.

Say this as an architect

Cursor is useful when it can see the ADR next to the principles and only report contradictions.

Appendix D — Glossary

Term	Meaning in this document
BFF	Backend for frontend. The only API the Angular app calls. Holds tokens. The browser gets a cookie.
Bounded context	A boundary where a word means one thing. Ordering's order is not the warehouse's pick list.
ADR	Architecture decision record. Context, decision, consequences, revisit trigger.
C4	Context, container, component, code. Most reviews stop at container, plus one component view of a hard path.
SLI / SLO	The measurement, and the target for that measurement over a window.
Error budget	The failure the SLO still allows. For 99.9 percent of a 30-day month, about 43 minutes.
Outbox	A row written in the same transaction as the business change, later published to a bus.
Saga	Several local transactions with a compensation or a retry. Not a synonym for 'we use events'.
Fitness function	An automated check that a design decision still holds.
Strangler	Replace a legacy system one journey at a time, with a way back, until the old path has no writers.
Anti-corruption layer	A translation at the edge so a foreign model does not leak into yours.
Modular monolith	One deployable, with module and data boundaries you enforce. Not 'a monolith we are ashamed of'.
Well-Architected	Microsoft's five pillars: reliability, security, cost, operations, performance. A review should say which one you are spending.
Managed identity	An Azure identity for the app itself, so SQL and Key Vault do not need a password in a config file.
Idempotency key	A client-supplied key that makes a retried create return the original order instead of a second one.

Appendix E — What this month will not make you

You will not be an enterprise architect, a security consultant, or a person who has operated a multi-region system through a real failover. You will not have read Kleppmann or the Hard Parts cover to cover. You will not have a right to a title. You will have a portfolio you can defend, a way of speaking in decisions, and a 90-day plan that does not start with a rewrite.

If a week collapses because of work or family, do not quit the sequence and do not double the hours the next day. Keep the artifact and the spoken answer. Slide the external reading to the weekend or drop it. The calendar is a scaffold. The ADRs are the point.

When you finish day 30, book a conversation with the strongest architect you know and ask them to spend 45 minutes trying to break ADR-001 and the partner API. A human who has carried a pager will find things this document and the model both missed. That conversation is the start of the job, not a graduation.

A last rule about numbers

Every peak, latency, retention, and availability figure in this document is a teaching assumption for Harbor, except the calendar arithmetic on a 30-day month and the published .NET support window. If you say them in a real review, label them. Then replace them with the measurement, the bill, or the official limit.